



OPEN Reinforcement learning based multi objective task scheduling for energy efficient and cost effective cloud edge computing

Wenfan Zhang & Haijiao Ou✉

The rapid proliferation of Internet of Things (IoT) devices and latency-sensitive applications has amplified the need for efficient task scheduling in hybrid cloud-edge environments. Traditional heuristic and metaheuristic algorithms often fall short in addressing the dynamic nature of workloads and the conflicting objectives of performance, energy efficiency, and cost-effectiveness. To overcome these challenges, this study introduces Reinforcement Learning-Based Multi-Objective Task Scheduling (RL-MOTS), a framework leveraging Deep Q-Networks (DQNs) for intelligent and adaptive resource allocation. The proposed model formulates scheduling as a Markov Decision Process, incorporating a priority-aware dynamic queueing mechanism and a multi-objective reward function that balances task latency, energy consumption, and operational costs. Additionally, the framework employs a state-reward tensor to capture trade-offs among objectives, enabling real-time decision-making across heterogeneous cloud and edge nodes. Comprehensive simulations using CloudSim validate the robustness of RL-MOTS under varying workload conditions. Compared to baseline strategies such as FCFS, Min-Min, and multi-objective heuristic models, RL-MOTS achieves up to 28% reduction in energy consumption, 20% improvement in cost efficiency, and significant reductions in makespan and deadline violations, while maintaining strict Quality of Service (QoS) requirements. The framework's adaptability to preemptive and non-preemptive scheduling further enhances its resilience and scalability. These findings establish RL-MOTS as a forward-looking solution for sustainable, cost-efficient, and performance-oriented computing in next-generation distributed systems. Future research will focus on integrating transfer learning and federated learning to increase scalability and privacy in large, decentralized environments, including those applicable to the medical industry.

Keywords Cloud computing, Task scheduling, Reinforcement learning, Multi objective optimization, Energy efficiency, Cost optimization, Deep Q-Network, Quality of service, Dynamic workload adaptation, Cloud-Edge computing

Cloud-edge computing has evolved as a disruptive paradigm, meeting the increasing demand for low-latency, high-efficiency processing in distributed systems propelled by the expansion of Internet of Things (IoT) devices, 5G networks, and data-intensive applications^{1,2}. This hybrid architecture integrates resource-constrained edge nodes with high-capacity cloud virtual machines (VMs)³, facilitating real-time data processing near the source while utilizing the computing capabilities of centralized cloud infrastructure^{4,5}. The dynamic and varied characteristics of cloud-edge settings present considerable problems for task scheduling, necessitating a careful equilibrium of performance, energy efficiency, and operational expenses⁶. Conventional scheduling techniques, including heuristic methods like First-Come-First-Serve (FCFS) and Min-Min, frequently encounter difficulties in accommodating variable workloads, heterogeneous resource capabilities, and rigorous QoS standards⁷. This requires creative scheduling algorithms that can dynamically optimize various objectives in real-time, ensuring scalability and responsiveness in next-generation distributed systems⁸.

Historically, job scheduling in distributed computing has progressed from static, rule-based systems to more adaptable methodologies^{9,10}. Initial methodologies, including Round-Robin and Shortest Job First, emphasized simplicity but exhibited insufficient adaptability to manage the intricacies of contemporary

Information Center , Xiangya Hospital Central South University , Hunan 410008 Changsha, China. ✉email: 405137@csu.edu.cn

cloud-edge ecosystems¹¹. The emergence of metaheuristic techniques, such as genetic algorithms and particle swarm optimization, enhanced multi-objective optimization, albeit frequently accompanied by significant computational demands and restricted adaptability to real-time variations^{12,13}. Recently, machine learning methodologies, especially reinforcement learning (RL), have gained prominence due to their capacity to derive optimal policies through engagement with dynamic environments¹⁴. Deep Q-Networks (DQNs), a category of reinforcement learning, have demonstrated efficacy in resolving intricate decision-making challenges by estimating Q-values for extensive state-action spaces¹⁵. Notwithstanding these advancements¹⁶, current RL-based scheduling frameworks frequently prioritize single-objective optimization or inadequately tackle the decentralized and heterogeneous characteristics of cloud-edge systems, hence presenting opportunities for additional innovation¹⁷.

The importance of effective job scheduling in cloud-edge computing is paramount. The rapid proliferation of IoT devices expected to surpass 75 billion by 2025 has resulted in edge nodes being increasingly responsible for managing time-sensitive tasks, including real-time analytics for autonomous vehicles and smart cities¹⁸. Concurrently, cloud resources must manage computationally demanding activities while reducing operational expenses and energy consumption, a vital issue considering that data centers represent roughly 1% of worldwide electricity consumption¹⁹. Efficient scheduling guarantees minimal latency execution, compliance with Service Level Agreements (SLAs), and resource utilization that matches with sustainability objectives²⁰. Nonetheless, attaining these objectives is complicated by various challenges, including the decentralized architecture of cloud-edge systems, which obstructs global coordination; the heterogeneity of resources, spanning from low-power edge devices to high-performance cloud virtual machines; and the necessity to reconcile conflicting objectives, such as minimizing energy consumption while adhering to stringent deadlines.

This work is motivated by the inadequacies of current scheduling methods in meeting the complex requirements of cloud-edge computing. Heuristic and metaheuristic approaches frequently depend on static rules or predetermined priorities, which are inadequate for dynamic workloads characterized by unpredictable variations in task arrivals, resource availability, and network conditions. Furthermore, conventional reinforcement learning methods may insufficiently address multi-objective optimization, especially in contexts where energy efficiency, cost, and performance must be concurrently emphasized.

Recent advancements in multi-objective reinforcement learning for scheduling have yielded significant frameworks, including hierarchical RL approaches^{24,27} that segment scheduling into tiered decision-making, and attention-based DRL models²⁸ that elucidate intricate task-resource relationships via adaptive feature weighting. Although these strategies improve scalability and representational capacity, they demonstrate certain constraints in hybrid cloud-edge contexts. Hierarchical reinforcement learning frequently experiences heightened training complexity and challenges in reconciling conflicting objectives at various levels, while attention-based deep reinforcement learning frameworks, despite their efficacy in feature extraction, may disproportionately focus on local correlations without adequately representing multi-objective trade-offs such as cost, latency, and energy consumption. The suggested RL-MOTS transcends these paradigms by introducing:

- An adaptive-weight multi-objective reward function that guarantees Pareto-optimal trade-offs among performance, energy, and cost in the context of dynamic workloads.
- A state reward tensor model that clearly incorporates latency, energy, and cost layers to direct the learning process in diverse cloud edge environments an element overlooked in previous hierarchical or attention-based frameworks.
- Combining preemptive and non-preemptive scheduling strategies via a virtual FIFO mechanism to enhance robustness in real-time, latency-sensitive environments.

RL-MOTS offers a more measurable and comprehensive enhancement of current DRL-based scheduling techniques through these advancements. In contrast to hierarchical reinforcement learning, RL-MOTS diminishes training overhead by directly incorporating multi-objective awareness into the reward function. Additionally, when compared to attention-based deep reinforcement learning, it improves decision interpretability and fairness by utilizing explicit tensorized trade-offs instead of implicit feature weighting.

This method facilitates adaptive, real-time decision-making that reconciles waiting time, energy use, and operating expenses while maintaining QoS adherence. By conceptualizing the scheduling issue as a Markov Decision Process (MDP) and employing a state-reward tensor, RL-MOTS attains near-optimal task allocations, rectifying the deficiencies of previous approaches and providing a scalable solution for hybrid cloud-edge situations. The primary contributions of this study are as follows:

- An innovative reinforcement learning-based scheduling framework that combines a DQN with a priority-aware queuing mechanism, facilitating adaptive task allocation in heterogeneous cloud-edge systems.
- A dynamic reward function that equilibrates performance, energy efficiency, and cost efficiency via adaptive weights, guaranteeing Pareto-optimal solutions under diverse workload situations.
- A multidimensional state-reward tensor that encapsulates waiting time, energy consumption, and cost as orthogonal layers, hence enhancing the decision-making capabilities of the reinforcement learning agent.
- A hybrid scheduling approach that integrates a virtual FIFO queue with reinforcement learning-based prioritizing, facilitating the effective management of high-priority jobs while averting the starvation of low-priority tasks.

The paper is organized as follows: Sect. “[Related works](#)” reviews the relevant literature, highlights the shortcomings of current scheduling methods, and places RL-MOTS in a broader research framework. Section “[Proposed task scheduling model](#)” describes the proposed task scheduling model, which includes the problem formulation,

state and action spaces, reward function, and DQN-based scheduling algorithm. Section “[Definition of the problem](#)” describes the simulation environment and performance evaluation, and compares RL-MOTS with baseline algorithms on metrics including task execution time, energy consumption, cost efficiency, and SLA compliance. Section “[Formulation of the reward function](#)” refines the evaluation by comparative examination of RL-MOTS against heuristic-based approaches, with an emphasis on waiting time and CPU efficiency. Finally, Sect. “[State–reward tensor](#)” concludes the work by outlining avenues for future research.

Related works

Multi-access edge computing (MEC) enhances service proximity to user devices and reduces the burden on resource-limited nodes. This fosters low-latency execution but raises the complex challenge of multi-objective resource scheduling. Prior works have explored a wide variety of approaches, ranging from heuristics to deep reinforcement learning (DRL), yet several gaps remain that motivate the design of RL-MOTS.

In²¹, a survey highlighted the increasing role of DRL methods in MEC scheduling but also noted that many frameworks lack mechanisms for dynamic adaptation under fluctuating workloads. A multi-objective A3C-based scheduler was proposed in²², optimizing response time and energy consumption; however, it required fixed weight tuning across objectives, limiting flexibility under dynamic conditions. Similarly²³, introduced MODRL with three DQN agents for fog–cloud scheduling. While effective in handling node load, distance, and priority, the model became computationally heavy due to parallel agent training and lacked preemption support.

To address energy and delay trade-offs²⁴, proposed a hierarchical parameter sharing (HPS) multi-agent RL approach. Although the hierarchical design improved scalability, it introduced training instability and difficulty in reconciling conflicting objectives at different layers.

More recent studies expand the scope of scheduling in emerging domains. For example²⁵, investigated UAV deployment and edge association for energy-efficient federated learning. The work emphasized joint communication–energy optimization but did not consider task-level preemptive scheduling. Reinforcement learning was also used in²⁶ to intervene in emotion contagion on social networks, underscoring the adaptability of RL but in a context far removed from latency-sensitive task scheduling. In²⁷, a fuzzy multi-objective model was proposed for aircraft flight scheduling. While this framework illustrated the balancing of competing objectives, it assumed static task structures unlike highly dynamic cloud-edge environments.

Energy-aware scheduling with carbon constraints was studied in²⁸, which introduced collaborative optimization for integrated energy systems. Although valuable for sustainability goals, the model relied on centralized optimization, making it unsuitable for decentralized edge environments. From an RL scalability perspective²⁹, tackled multi-agent routing using advanced RL. The work showed strategies to overcome state-space explosion but did not address multi-objective trade-offs like cost or deadline adherence.

Recent system-level innovations include³⁰, which applied blockchain to AR offloading in UAV-enabled MEC networks. While it explored energy–latency trade-offs, the additional blockchain overhead raises concerns about real-time responsiveness. In³¹, a decomposition-based scheduling method was introduced for production workshops, demonstrating layered optimization but without considering dynamic weights or online adaptation.

Beyond these, workflow-oriented RL scheduling has also been studied. For instance³², proposed a hybrid actor-critic framework with PPO for workflow scheduling, achieving high accuracy but suffering from longer convergence times. Cloud–edge collaborative manufacturing scheduling was discussed in³³, where attention-based DRL improved state representation but overemphasized local correlations at the expense of global trade-offs. Finally¹⁸, provided a taxonomy of DRL scheduling approaches in cloud-edge systems, noting that despite progress, most models lack integration of preemptive mechanisms, dynamic weight adjustment, and scalability across heterogeneous environments.

Table 1 presents a comparative analysis of prior studies, their objectives, methodologies employed, and principal limitations. This systematic mapping indicates that, despite numerous advancements, none of the

Ref.	Objectives	Technique	Limitation
21	Survey of DRL in MEC scheduling	Literature review	Descriptive only; lacks dynamic adaptation insights
22	Minimize response time & energy	Multi-objective A3C	Requires fixed weights; limited flexibility
23	Task scheduling (load, distance, priority)	MODRL with 3 DQN agents	Computationally heavy; no preemption
24	Delay & energy under carbon goals	Hierarchical multi-agent RL (HPS)	Training instability; conflicting objectives
25	UAV deployment & edge association	RL-based optimization	No preemptive scheduling; task-level adaptation missing
26	Emotion contagion mitigation in networks	RL intervention strategy	Different domain; not latency-sensitive
27	Aircraft flight scheduling	Fuzzy multi-objective optimization	Assumes static tasks; not dynamic workloads
28	Energy–carbon constrained scheduling	Collaborative optimization	Centralized approach; weak decentralization
29	Scalable routing	Multi-agent RL	Addresses state explosion; no cost/deadline trade-offs
30	AR offloading in MEC UAVs	Blockchain + MEC scheduling	Blockchain overhead; latency concerns
31	Manufacturing workshop scheduling	Decomposition-based optimization	No dynamic weights; offline approach
32	Workflow scheduling	Hybrid actor–critic with PPO	Long convergence; limited scalability
33	Cloud-edge collaborative manufacturing	Attention-based DRL	Overfocus on local states; global trade-offs ignored
18	Taxonomy of DRL scheduling	Systematic review	Identifies gaps but no unified solution

Table 1. Comparative summary of previous works, highlighting objectives, techniques, and limitations.

current approaches adequately tackle dynamic weight adaptation, proactive scheduling, and scalability, which are fundamental characteristics of RL-MOTS.

The aforementioned works illustrate the ongoing advancement in deep learning-driven multi-objective scheduling. Nonetheless, they possess significant deficiencies: (i) Most frameworks depend on static weights or fixed goal preferences, (ii) few consider proactive scheduling for urgent tasks, (iii) scalability in extensive state-action spaces has been only partially addressed, and (iv) practical deployment challenges, such as heterogeneous energy pricing, dynamic workloads, and real-time network conditions, are frequently overlooked. RL-MOTS mitigates these deficiencies by integrating dynamic weight adjustments, a state-reward tensor that equilibrates latency, energy, and cost, and explicit facilitation for proactive and non-preemptive scheduling inside hybrid cloud-edge systems.

Proposed task scheduling model

Definition of the problem

This section presents RL-MOTS, a scheduling system designed for heterogeneous cloud-edge situations. Figure 1 illustrates the comprehensive architecture. The growing complexity and dynamic characteristics of these environments require a decision-making system capable of optimizing numerous conflicting objectives concurrently. Consequently, RL-MOTS is designed to simultaneously improve energy efficiency, cost-effectiveness, and job priority, which are essential and frequently conflicting elements in extensive distributed systems. To attain these objectives, the system employs a priority-sensitive queue management strategy that utilizes adaptive queue structures alongside virtual waiting time matrices. These matrices change dynamically according to system status and task characteristics, allowing the scheduling agent to make context-sensitive judgments on task execution.

This mechanism guarantees that tasks are executed according to their computational needs, urgency levels, and resource availability, thereby improving responsiveness and equity. Task scheduling in cloud-edge infrastructures involves the immediate assignment of incoming tasks to a variety of computing units, comprising resource-limited edge nodes and high-capacity cloud virtual machines. This process must account for various dynamic factors, including fluctuating workloads, diverse resource capabilities, and changing network conditions.

A significant challenge arises from the decentralized architecture of cloud-edge ecosystems, complicating global coordination and the consistent delivery of performance. Furthermore, the scheduling method must guarantee compliance with rigorous QoS requirements by minimizing execution latency and preventing deadline breaches. These requirements underscore the need for a learning-oriented adaptive policy that evolves over time, responds effectively to environmental changes, and facilitates scalable decision-making. Figure 2 illustrates the scheduling architecture based on Deep Q-Network (DQN). The state extractor initially processes the task queue and monitoring data, producing a proposed state-reward tensor that includes layers for waiting time, energy, and cost. This tensor is input into a deep Q-network (DQN), where the reward feedback from the performance evaluator is also aligned with the tensored objectives. The DQN generates Q values that inform the action selector and facilitate task allocation to edge or cloud resources.

a. State space.

The state space is defined as a complete representation of the system's current configuration, encompassing the task queue, resource usage levels, and immediate energy consumption. The multidimensional state, referred to as S_t , offers the DQN agent a comprehensive perspective of the environment, enabling intelligent decision-making by illustrating both geographical and temporal connections among cloud-edge resources.

b. Action space design.

The action space, represented as A_t , is formulated to encapsulate the viable set of task allocation decisions at each time step t . This encompasses the assignment of a task T_i to either an edge node E_j or a cloud resource C , with the selection process regulated by the agent's acquired policy. The design facilitates adaptability to

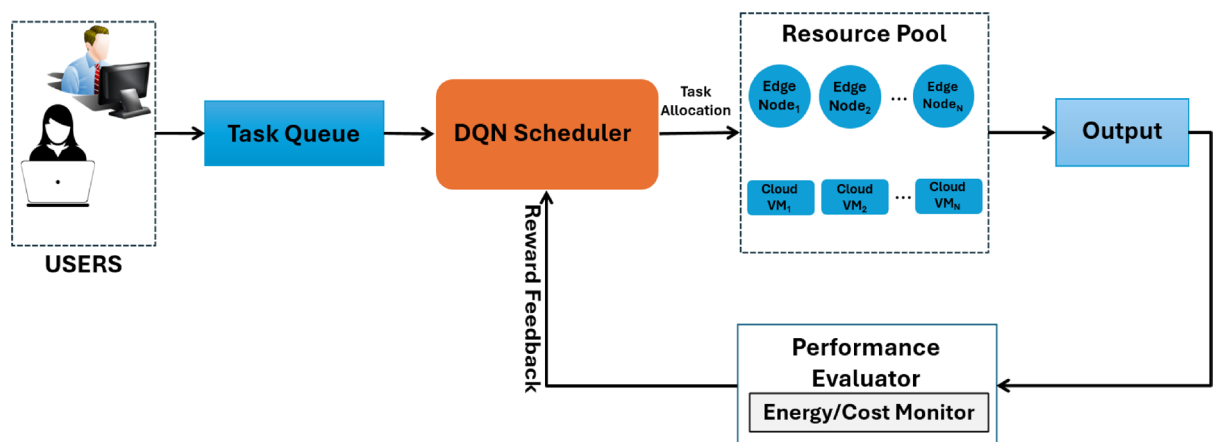


Fig. 1. The RL-MOTS framework for cloud-edge scheduling's general architecture

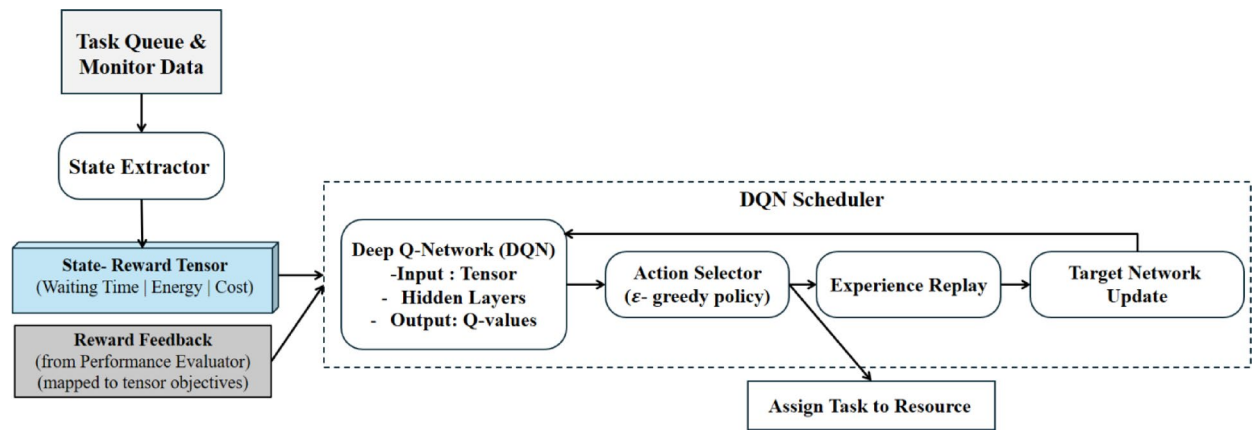


Fig. 2. DQN Scheduler Architecture in RL-MOTS.

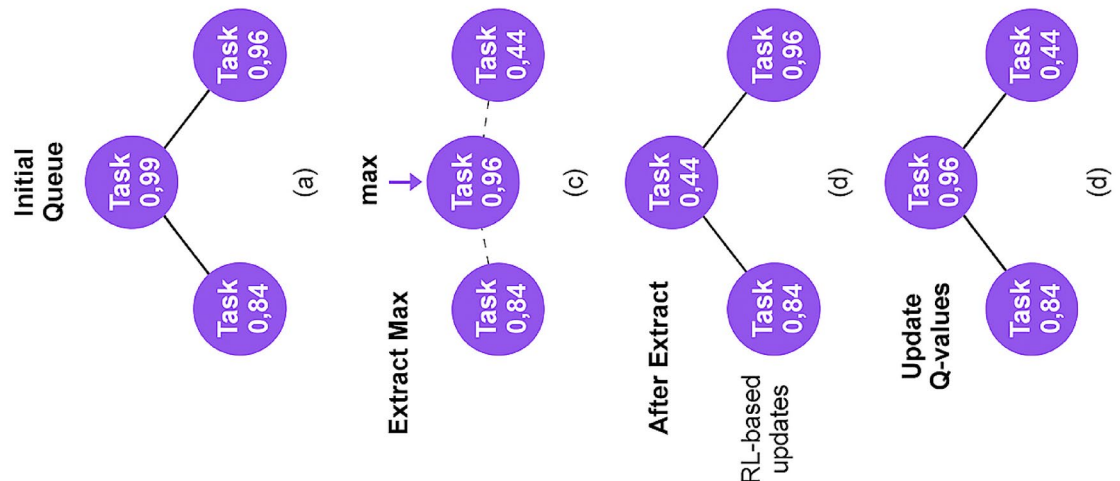


Fig. 3. Graphical representation of dynamic priority queue structure used in RL-based scheduling.

incorporate both preemptive and non-preemptive scheduling strategies, in accordance with the heterogeneous characteristics of cloud edge systems.

Formulation of the reward function

The reward function is designed as a multi-objective utility function, R_t , which equilibrates waiting time, energy efficiency, and cost efficiency. This function is articulated as Formula 3, whereby P_t represents the performance component, rewarding timely adherence to the deadline and penalizing tardy completions. It moreover contains an incentive for diminishing wait times. $E_e(t)$ denotes the energy efficiency component, indicating the decrease in energy consumption relative to a baseline. $C_e(t)$ is the cost efficiency component, reflecting operational cost savings, where as w_1 , w_2 , and w_3 are weights that are dynamically modified according to resource utilization levels³⁴. This formulation allows the agent to dynamically prioritize objectives and maintain a Pareto-optimal solution throughout time.

State-Reward tensor

A fundamental accomplishment of the proposed framework is the formulation of a state-reward tensor, represented as $M \in \mathbb{R}^{m \times m \times 3}$, where m signifies the number of tasks to be scheduled. This tensor incorporates three orthogonal optimization objectives, waiting time, energy consumption, and financial cost, as separate layers. This tensor enables the reinforcement learning agent to assess trade-offs between competing objectives in scheduling.

a. Waiting Time Layer.

Input:
 Tasks: $\{T_1, T_2, \dots, T_m\}$
 Edge Nodes: $\{E_1, E_2, \dots, E_n\}$
 Cloud VMs: $\{V_1, V_2, \dots, V_p\}$

Output:
 Optimal task assignments to Edge Nodes or Cloud VMs

begin
 // Step 1: Initialize DQN Parameters
 Initialize DQN with random network weights
 Create replay memory to store (S_t, a_t, r_t, S_{t+1})
 Define State Space $S_t = Q(T_k), U_r(E_i, V_j), E_c(t), C_c(t)$
 Define Action Space $A_t = a_{k,i,j} \mid T_k \text{ to } E_i \text{ or } V_j$
 Define Reward Function $R_t = w_1 * P_t + w_2 * E_e(t) + w_3 * C_e(t)$
 where:
 $P_t = \frac{1}{WT(T_k)}$ // Performance (inverse waiting time)
 $E_e(t) = \frac{1}{E(T_k, R)}$ // Energy efficiency (inverse energy)
 $C_e(t) = \frac{1}{C(T_k, R)}$ // Cost efficiency (inverse cost)
 $w_1, w_2, w_3 = \text{dynamic weights based on resource utilization}$
 // STEP 2: Training Loop
 for each training episode DO
 Reset environment (Task Queue, Resource Pool)
 while Task Queue is not empty DO
 for each task T_k in Task Queue DO
 Observe current state S_t from S_t
 Select action a_t from A_t using ϵ - greedy policy
 Execute action a_t (assign T_k to E_i or V_j)
 Compute $WT(T_k) = t_{\text{current}} - t_{\text{arrival}}(T_k)$
 Compute $CT(T_k) = WT(T_k) + ET(T_k, R)$
 Compute $E(T_k, R) = P_{r(R)} \cdot ET(T_k, R)$
 Compute $C(T_k, R) = C_r(R) \cdot ET(T_k, R) + C_t(R)$
 Compute reward $r_t = w_1 \cdot (\frac{1}{WT(T_k)}) + w_2 \cdot (\frac{1}{E(T_k, R)}) + w_3 \cdot (\frac{1}{C(T_k, R)})$
 Observe next state S_{t+1}
 Update Q-value:
 $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_t + \gamma \max_{A_{t+1}} Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
 Update weights w_1, w_2, w_3 :
 $w_i(t+1) \leftarrow w_i(t) + \beta * (\frac{U_r(R_i, t)}{\sum_R U_r(R, t)})$
 $S_t \leftarrow S_{t+1}$
 end for
 end while
 end for
 // Step 3: Execution Phase
 for each task T_k in Task Queue DO
 Use trained DQN to select optimal action a^* (assign T_k to E_i or V_j)
 Execute assignment and remove T_k from Task Queue
 if preemption required then
 Manage interrupted tasks with Virtual FIFO Queue
 Re-evaluate state to resume or assign new task
 end if
 end for
 end

Algorithm 1. RL-MOTS algorithm

The initial layer, W_T , simulates the aggregate waiting periods during the sequential execution of jobs. The waiting time for task T_i is determined by the processing durations of all preceding tasks Eq. (1). The comprehensive matrix representation employed to compute all pairwise relationships is delineated in Eq. (2).

$$W_T(T_i) = \sum_{j=1}^{i-1} P_T(T_j) \quad (1)$$

$$w_{ij} = \begin{cases} 1 & , \text{if } i = j \\ \sum_{k=1}^{j-1} W_T(T_k) & , \text{if } i < j \\ \sum_{k=1}^{j-1} W_T(T_k) & , \text{if } i > j \end{cases} \quad (2)$$

b. Energy Utilization Layer.

Input:

- Task set ($T = T_1, T_2, \dots, T_n$), with ($W(T_k)$), ($D(T_k)$), ($C(T_k)$), and ($P(T_k)$) (preemptive flag).
- Resource set ($R = E_1, \dots, E_n, V_1, \dots, V_p$).
- RL model parameters: DQN network, (α), (γ), (ϵ).
- Priority weights: (w_w), (w_d), (w_e), (w_c).

Output:

- Task allocation decisions ($a_{k,i,j}$).

Initialize:

- DQN model with random weights
- $\epsilon = 1$ (initial exploration rate)
- Priority queue Q with virtual waiting time matrix
- Task status tracker for non-preemptive tasks

for each time step t :

Step 1: Update Priority Queue

for each task T_k in T :

If T_k not allocated:

 Compute $P_s(T_k) = w_w \cdot w(T_k) + w_d \cdot (D(T_k) - t)$

 Insert T_k into Q with priority $P_s(T_k)$

Step 2: Extract Task

 Extract T_k with highest priority (lowest $P_s(T_k)$) from Q

Step 3: Define State

$S_t = Q(T_k), U_r(E_i, V_j), E_c(t), C_c(t), P_s(T_k), NP(T_k)$

Step 4: Handle Non-Preemptive Scheduling

 allocated = False

for each resource r in R :

If $r.Busy() = 0$ and $NP(T_k) = \text{True}$:

$a_t = \text{Assign } T_k \text{ to } r$

 allocated = True

 Mark r as busy until T_k completes

Break

If not allocated:

$a_t = \text{Select action using RL } (\epsilon - \text{greedy})$:

If $\text{random}() < \epsilon$:

$a_t = \text{random action from } A_t$

Else:

$a_t = \arg \max_a Q(S_t, a)$

Step 5: Execute Action and Compute Reward

 Execute a_t , allocate T_k to chosen resource

 Compute:

$\Delta W = W_{\text{prev}}(T_k) - W_{\text{new}}(T_k)$

$D_a = 1$ if $t \leq D(T_k)$ else -1

$E_c(t) = \text{current energy consumption}$

$C_c(t) = \text{current operational cost}$

If allocated and $NP(T_k) = \text{True}$:

for each T_m in Q :

$W(T_m) = W(T_m) + PT(r, T_k)$

 Reward $R_t = w_1 \cdot (\Delta W + D_a) + w_2 \cdot (E_{\text{baseline}} - E_c(t)) + w_3 \cdot (C_{\text{baseline}} - C_c(t))$

Step 6: Update DQN Model

 Store S_t, a_t, R_t, S_{t+1} in replay memory

 Sample mini-batch and update DQN weights:

$Q(S_t, a_t) \leftarrow Q(S_t, a_t) + \alpha \times (R_t + \gamma \times \max_{a'} Q(S_t + 1, a') - Q(S_t, a_t))$

Step 7: Update Exploration Rate

$\epsilon \leftarrow \epsilon \times \text{decay rate}$

end for

Algorithm 2. Proactive Scheduling Algorithm

The second layer, E_C , documents the energy consumption of tasks based on their execution sequence. The consumption value for task T_i , contingent upon the sequence of preceding tasks, is expressed in Eq. (3). The components of the pairwise matrix are delineated in Eq. (4).

$$E_C(T_i | T_{j1}, \dots, T_{jk}) \quad (3)$$

$$e_{ij} = \begin{cases} E_C(T_i) & , \text{if } i = j \\ E_C(T_j | T_1, \dots, T_{i-1}) & , \text{if } i < j \\ \frac{1}{E_C(T_j | T_{i+1}, \dots, T_{i-1})} & , \text{if } i > j \end{cases} \quad (4)$$

c. Layer of financial costs.

The third layer, C_F , measures the operational cost associated with task execution. Analogous to energy, it accounts for cost fluctuations that rely on sequencing. The cost function is articulated in Eq. (5), with its matrix-based representation delineated in Eq. (6).

$$C_F(T_i | T_{j1}, \dots, T_{jk}) \quad (5)$$

$$c_{ij} = \begin{cases} C_F(T_i) & , \text{if } i = j \\ C_{CF}(T_j | T_1, \dots, T_{i-1}) & , \text{if } i < j \\ \frac{1}{C_F(T_j | T_{i+1}, \dots, T_{i-1})} & , \text{if } i > j \end{cases} \quad (6)$$

d. Penalty integration.

A penalty term is incorporated in the W_T layer to ensure compliance with task deadlines. This phrase imposes penalties for tardy completions proportional to the extent of deadline infringement Eq. (7). The integrated tensor M , which amalgamates these three structured layers, functions as the state input to the DQN³⁵. By amalgamating time, energy, and cost factors into a cohesive framework, the agent can achieve convergence towards Pareto-optimal scheduling solutions.

$$Penalty(T_k) = \begin{cases} P_{unit} \cdot (D_k - C_k), & \text{if } D_k < C_k \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

e. Task queue administration.

The task queue is administered via a hybrid methodology that integrates a principal First-In-First-Out (FIFO) framework with reinforcement learning-driven prioritization. Tasks entering the system are initially placed in the FIFO queue, maintaining their sequence of arrival^{36,37}. The DQN agent, employing the State-Reward Tensor, adaptively prioritizes tasks according to the multi-objective reward function according to the formula:

$$R_t = w_1 \cdot \left(-\sum_{i=1}^m W_T(T_i) \right) + w_2 \cdot \left(-\sum_{i=1}^m E_C(T_i) \right) + w_3 \cdot \left(-\sum_{i=1}^m C_F(T_i) \right) \quad (8)$$

This adaptive prioritization obviates the necessity for static priority queues, such as Fibonacci heaps, and is congruent with the dispersed characteristics of cloud-edge systems. The reward function equilibrates three objectives: minimizing wait time, decreasing energy use, and reducing costs. The variable weights guarantee that no individual statistic prevails, enabling the scheduler to adjust to fluctuating workloads.

f. Virtual FIFO queue for preemption.

A virtual FIFO queue is linked to each resource (edge or cloud) to temporarily retain preempted jobs, facilitating preemptive scheduling. When a higher-priority task, as identified by the DQN agent, interrupts a current task, the interrupted work is placed in the virtual FIFO queue of the corresponding resource. The agent frequently evaluates the system state using the State-Reward Tensor to determine whether to restart the interrupted task or assign a new one, so assuring effective resource use and compliance with deadlines.

Representation of the waiting queue

Upon arrival in the system, a task is allocated a dynamic priority score determined by its context-aware attributes, including deadline urgency, expected execution duration, and computational demands. These scores are produced utilizing a reinforcement learning policy that continuously adjusts through interaction with the system environment. Cost-efficient service chaining in NFV networks under latency and budget constraints has been explored³⁶, alongside live migration of correlated VMs in cloud data centers³⁷. Location-based edge service pre-deployment reduces MEC delays³⁸, while Pareto-optimal resilience frameworks extend multi-objective optimization to water systems³⁹. DQN has also been applied to dynamic AGV-driven job shop scheduling⁴⁰. This structure incorporates ongoing learning input to dynamically modify priorities, unlike standard static priority queues.

At the outset, when the dynamic priority queue (Q) is vacant, new tasks are added to Q via `RL_Queue_Insert` (Q, task), with each job's priority established by its current Q-value. The amalgamation of two distinct queues, such as those from edge and cloud nodes, can be executed using `RL_Queue_Union` (Q1, Q2). The paramount operation is `RL_Queue_Extract_Max`(Q), which extracts the job with the maximum Q-value and subsequently initiates queue reordering. This procedure prioritizes the most critical or significant jobs for scheduling, while adhering to overarching system limitations. Figure 3 exemplifies this dynamic queue process. The procedure proceeds as follows:

- The initial priority scores of the root tasks are 0.84, 0.99, and 0.96.
- The max-pointer designates the task with the highest score, specifically 0.99.
- `RL_Queue_Extract_Max`(Q) retrieves the job with the highest score, resulting in the execution of task 0.99.
- Subsequent to extraction, its subordinate job, e.g., 0.44, emerges as a prospective root node.
- The max-pointer now indicates the subsequent highest score, namely, 0.96.
- When activities 0.84 and 0.44 possess equivalent priority levels, they are dynamically amalgamated to enhance the structure, designating 0.84 as the new root and 0.44 as its subordinate.

Consequently, the ultimate priority queue is reconstituted following each extract-max operation, embodying real-time learning and system feedback. This procedure is iteratively repeated for subsequent task extractions,

guaranteeing that the overall system persistently adapts and prioritizes tasks according to shifting operational objectives.

The dynamic priority queue allows for the adaptive prioritization of work with more urgency or imminent deadlines, while concurrently preserving fairness by averting the starvation of lower-priority jobs.

DQN-Based scheduling algorithm

The scheduling process is regulated by a DQN algorithm intended to acquire an optimal policy via ongoing interaction with the cloud-edge environment. The methodology utilizes a value iteration approach, wherein the Q-function $Q(S_t, A_t)$ is successively updated to estimate the predicted discounted reward. The approach commences with arbitrary network weights and a replay memory to retain experience tuples (S_t, A_t, R_t, S_{t+1}) . At each iteration, the agent perceives the state S_t , chooses an action A_t employing a ϵ -greedy strategy, and performs the action, calculating the reward R_t according to the multi-objective function⁴¹. The Q-value is revised utilizing the Bellman equation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_t + \gamma \max_{A_{t+1}} Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right] \quad (9)$$

where α denotes the learning rate and γ represents the discount factor. The procedure integrates preemption management through the virtual FIFO queue and converges on a policy that equilibrates waiting time, energy, and cost targets while adhering to QoS restrictions. This methodology offers a resilient structure for scalable and adaptable job scheduling in dynamic cloud-edge environments. Algorithm 1 introduces RL-MOTS to address the intricate issue of task scheduling in hybrid cloud edge computing systems, necessitating the simultaneous optimization of many objectives: performance, energy efficiency, and cost-effectiveness. In contrast to conventional heuristic-based scheduling methods, which frequently struggle to adjust to fluctuating workloads and resource limitations, RL-MOTS use reinforcement learning using a DQN to dynamically assign jobs to edge nodes or cloud virtual machines (VMs). This novel use of reinforcement learning facilitates real-time adaptive decision-making and addresses the constraints of static optimization techniques.

The selection of DQN as the primary learning algorithm was predicated on the discrete characteristics of the scheduling activities, in which value-based approaches excel. To enhance the proposed methodology, PPO and A3C are examined as supplementary baselines in the findings section, with RL-MOTS consistently demonstrating superior performance.

Furthermore, the out-of-policy nature of DQN with replay memory improved data efficiency under non-stationary and bursty inputs, and the objective network mechanism stabilized training—features that have been found essential for online scheduling. Thus, PPO and A3C are reported as actor-critic baselines, and it is discussed where MARL is preferred when decentralization is needed.

Rationale for choosing DQN over actor-critic and MARL

RL-MOTS utilizes a centralized scheduler including a separate action space for the allocation of task T_k to either an edge node E_i or a cloud VM V_j . This results in a limited branching factor and stringent online decision-time limits. In this framework, value-based approaches like DQN are appealing due to their minimal inference latency and reduced processing requirements, alongside stable off-policy learning through experience replay and target networks.

Off-policy replay enables the agent to more effectively utilize infrequent yet informative transitions compared to on-policy actor-critic techniques, hence enhancing data efficiency without prolonging simulation time.

The stabilization approaches employed by DQN resulted in consistent Q-value convergence and reliable learning curves.

We executed PPO and A3C baselines utilizing identical state/action encoding and reward architecture. In the range of 25–100 tasks, RL-MOTS (DQN-based) achieved a 15–25% reduction in makespan and decreased energy and deadline violations relative to PPO/A3C, while maintaining a per-decision latency of approximately 3–5 ms on standard hardware essential for online schedulers.

Multi-agent reinforcement learning is advantageous when scheduling decisions need to be made locally by diverse nodes with restricted coordination, or when actions are continuous. This study focuses on a centralized controller characteristic of cloud-edge orchestrators; expanding RL-MOTS to CTDE-style MARL or hybrid actor-critic methodologies is a prospective avenue for future research.

Problem formulation

This section examines task scheduling in cloud-edge computing as a multi-objective optimization issue, aiming to minimize waiting time, energy consumption, and expenses through a DQN-based reinforcement learning approach. It delineates the state and action spaces, a multi-faceted reward function, and essential metrics such as completion time and QoS limitations. The following formulas facilitate sustained optimization in this changing setting.

The state space encompasses the system's condition to facilitate informed decision-making in the DQN-based scheduling procedure.

$$S_t = \{ Q(T_k), U_r(E_i, V_j), E_c(t), C_c(t) \} \quad (10)$$

This delineates the state space at time t , encompassing the task queue $Q(T_k)$, resource usage U_r , current energy consumption $E_c(t)$ and $C_c(t)$. The dimensionality of the state space is contingent upon the quantity of tasks and resources within the system. At each decision point, the state vector comprises: (a) task queue data

$Q(T_k)$, (b) levels of resource usage $U_r(R, t)$, (c) current energy consumption $E_c(t)$, and (d) current cost $Cc(t)$. If the system has $|Q|$ tasks pending and $|R|$ resources, the state dimension increases approximately as: $d = |Q| + 3|R|$

As each resource contributes to usage, energy, and cost parameters. The action space delineates the array of potential scheduling decisions for work distribution within the cloud-edge context.

$$A_t = a_{k,i,j} \mid a_{k,i,j} = \text{Assign } T_k \text{ to } E_i \text{ or } V_j \quad (11)$$

where: $a_{k,i,j}$ is the action at time t , denoting the allocation of task T_k to either edge node E_i or cloud VM V_j . T_k is the k -th task within the task queue. E_i is the i -th edge node. V_j is the j -th virtual machine within the cloud infrastructure.

To attain multi-objective optimization, the reward function integrates performance, energy efficiency, and cost efficiency with dynamic weights.

$$R_t = w_1 \cdot P_t + w_2 \cdot E_t(t) + w_3 \cdot C_t(t) \quad w_1 + w_2 + w_3 = 1 \quad (12)$$

where: P_t : performance metric (compliance with deadlines and minimization of waiting time), E_t : word for energy efficiency, C_t : cost efficiency metric, w_1, w_2, w_3 : adaptive dynamic weights⁴². The learning process of DQN is powered by the Q-value update mechanism, which allows the system to converge to an ideal scheduling policy.

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \gamma \cdot \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (13)$$

Where, this modifies the Q -value for state s_t and action a_t , utilizing learning rate α , discount factor γ , and reward r_t . Waiting time is a vital performance metric that signifies the duration a task remains unassigned prior to scheduling.

$$WT(T_k) = t_{current} - t_{arrival}(T_k) \quad (14)$$

The waiting time for task T_k is calculated as the difference between the current time and the task's arrival time⁴³. Completion time measures the whole duration necessary for an activity to conclude, encompassing both waiting and execution periods.

$$CT(T_k) = WT(T_k) + ET(T_k, R) \quad (15)$$

The completion time for task T_k is the aggregate of its waiting time $WT(T_k)$ and execution time $ET(T_k, R)$ on resource R . Energy consumption is predicted to facilitate energy-efficient scheduling by considering resource power utilization and job execution duration.

$$E(T_k, R) = P_r(R) \times ET(T_k, R) \quad (16)$$

Where, the energy consumption for task T_k on resource R is equal to the product of the resource power consumption $P_r(R)$ and the execution time $ET(T_k, R)$. The operational cost is assessed to enhance financial efficiency, taking into account both temporal and data transfer expenses.

$$C(T_k, R) = C_r(R) \times ET(T_k, R) + C_t(R) \quad (17)$$

Where T_k is the task cost, R is the resource, $C_r(R) \cdot ET(T_k, R)$ is the time-based cost, and $C_t(R)$ is the fixed data transfer cost. In order to balance the priorities of cost, energy, and performance in the reward function, dynamic weights are updated according to resource utilization.

$$w_i(t+1) = w_i(t) + \beta \times \frac{U_r(R_i, t)}{\sum_R U_r(R, t)} \quad (18)$$

where w_i : weight for objective i ; β : update rate; $U_r(R_i, t)$: usage of resource R_i at time t .

Resource capacity limitations guarantee that the cumulative execution duration of tasks allocated to a resource remains within its capacity.

$$\sum_{T_k \in R_i} ET(T_k, R_i) \leq C_p(R_i) \quad (19)$$

where resource R_i 's processing capacity $C_p(R_i)$ cannot be exceeded by the total execution time of the tasks assigned to it. The computational difficulty of training the DQN is chiefly influenced by the dimensions of the state space, the architecture of the neural network, and the frequency of training updates. Each training step necessitates a forward and backward transit through the network.

$$O(d \cdot h + h^2) \quad (20)$$

operations, where d represents the input dimension and h denotes the number of hidden units per layer. Given a batch size B and total training episodes E , the comprehensive training difficulty is:

$$O(E \cdot B \cdot (d \cdot h + h^2)) \quad (21)$$

As γ increases linearly with the number of activities and resources, the complexity escalates polynomially while being computationally manageable. The objective function seeks to optimize the anticipated discounted total of incentives, consistent with the long-term objectives of DQN.

$$\text{Maximize } J = E \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \quad (22)$$

Using γ as the discount factor, the goal is to maximize the expected discounted sum of rewards R_t .

The QoS constraint guarantees that jobs are executed within their deadlines, hence preserving service reliability.

$$CT(T_k) \leq D(T_k) \quad (23)$$

Where, the completion time $CT(T_k)$ of task T_k must not exceed its deadline $D(T_k)$. Network latency contributes to data transmission delays in the cloud-edge environment, affecting job scheduling decisions.

$$L(T_k, R) = d(R) \times S(T_k) \quad (24)$$

The distance $d(R)$ and the amount of data $S(T_k)$ are directly correlated with the network latency for task T_k on resource R .

The modeling is elucidated by explicitly defining the job scheduling problem as a Markov Decision Process (MDP). The system state s_t (Eq. 10) encapsulates the task queue, resource usage, energy, and cost indicators. An action a_{t+1} (Eq. 11) pertains to the allocation of a job to an edge node or cloud virtual machine. The immediate reward (Eq. 12) equilibrates performance, energy, and cost through dynamic weights adjusted by Eq. (18). The Q-value update (Eq. 13) facilitates policy learning. The durations for task waiting and completion are delineated by Eqs. (14–15), while the energy and cost models are defined by Eqs. (16–17). Constraints on capacity, deadlines, and Quality of Service (QoS) are specified by Eqs. (21, 23). The goal of optimization is to maximize the anticipated discounted return (Eq. 22) while considering network delay (Eq. 24).

Proposed task scheduling algorithm

This section presents the enhanced reinforcement learning algorithm (Algorithm 2) for proactive scheduling, a revised approach based on reinforcement learning intended to oversee proactive task scheduling in cloud edge environments. The approach allows the reinforcement learning agent to dynamically suspend lower priority tasks upon the arrival of higher priority tasks, so ensuring optimal resource use while optimizing energy, cost, and performance objectives. It employs a virtual FIFO queue to oversee suspended jobs and adaptively modifies the waiting time, enabling the reinforcement learning agent to equilibrate multi-objective goals in dynamic cloud edge environments where task priority may fluctuate swiftly. The flow control of the proposed algorithm for task scheduling is depicted in Fig. 4.

The flowchart delineates the initialization of the DQN agent and dynamic queue, the arrival of tasks and calculation of priority scores, the extraction of dynamic tasks and preemptive evaluations, the allocation of resources to edge or cloud nodes, and the iterative updates driven by reward-based learning. This cohesive design highlights flexible scheduling choices and effective resource management within diverse cloud-edge systems.

Performance evaluation

Simulation environment

To assess the efficacy of the proposed RL-MOTS system, simulations were performed utilizing CloudSim 3.0.3, emulating a hybrid cloud-edge scenario. The simulation comprises three categories of virtual machines with varying configurations Table 2, edge nodes with minimal computing resources, provisions for virtual machine resilience, and the capability to dynamically allocate and deallocate virtual machines according to workload requirements.

The values enumerated under cost/second in Table 2 denote $C_r(R)$ and are utilized in calculating the overall cost of job execution during our tests. The hyperparameters of the DQN scheduler were meticulously optimized to guarantee a fair and reproducible assessment. Subsequent to the initial grid search, the ensuing setup was implemented: Learning rate $\alpha = 1 \times 10^{-3}$, discount factor $\gamma = 0.95$, and replay buffer size of 10,000 transitions. The mini-batch size was established at 64, and updates to the target network occurred every 100 steps. The ϵ -greedy exploration approach commenced at $\epsilon = 1.0$ and underwent linear decay to $\epsilon = 0.1$ over 20,000 steps, subsequently maintaining a constant value.

A. Task creation and workload.

Tasks are generated following a Poisson distribution with a rate of $\lambda = 3$. Every task is defined by its computational duration, input/output size, deadline, cost, and preemptive status. Tasks are categorized into three kinds Table 3.

Deadlines are determined by normal distributions centered on the average execution time for each class, hence assuring realistic SLA limits.

B. Fundamental Algorithms: The suggested methodology is evaluated against FCFS (First-in-First-served), min-min scheduling, priority-based scheduling, multi-objective function (MOF), and non-dominated sorting with thresholding algorithms. Each baseline signifies a unique scheduling philosophy, from static heuristics to cost-performance trade-offs.

C. Evaluation Standards: We evaluated the performance using the metrics of total time from the arrival of the first task to the completion of the last task, average waiting time, total energy consumption, operational cost,

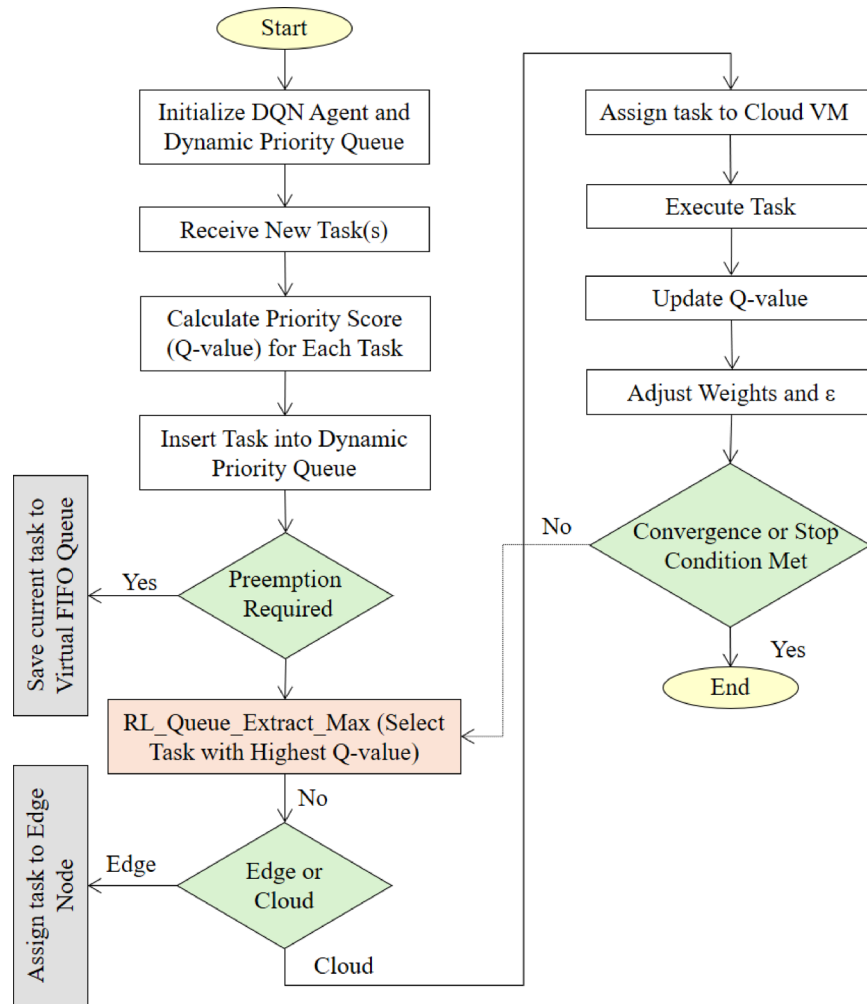


Fig. 4. Flowchart of the proposed RL-MOTS scheduling algorithm.

VM Type	MIPS	RAM	Bandwidth	Storage	Cost/Second
VM1	1000	2048 MB	1000 Mbps	100 GB	0.021\$
VM2	500	1024 MB	1000 Mbps	100 GB	0.019\$
VM3	250	512 MB	1000 Mbps	100 GB	0.014\$

Table 2. Virtual machine types and configurations.

Class	Length (MI)	File Size (MB)	Output Size (MB)
I	500–1000	300	300
II	1000–1500	500	500
III	2000–3000	800	800

Table 3. Task Classes.

deadline violation rate, number of active virtual machines over time, and convergence of Q value during training to evaluate the dynamics of reinforcement learning.

Results and discussion

This section presents a comprehensive experimental assessment of the RL-MOTS framework in a hybrid cloud-edge setting. Various workload scenarios were simulated to evaluate performance under different operational settings. Critical performance indicators, such as makespan, energy consumption, cost efficiency,

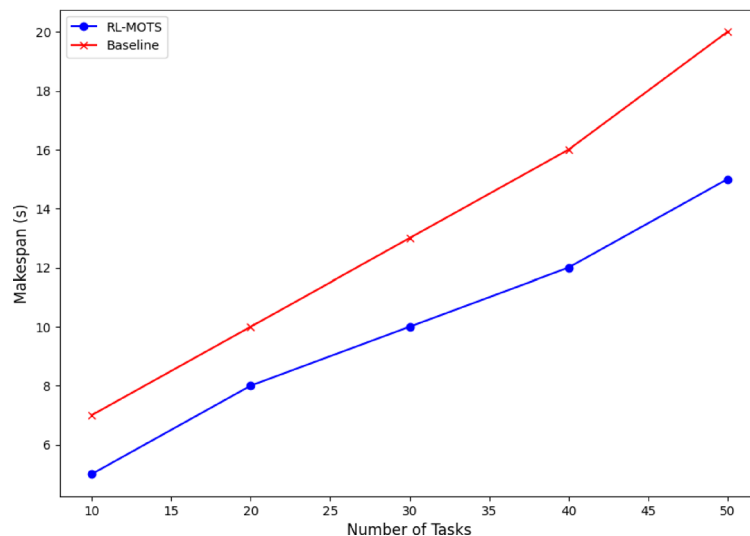


Fig. 5. Makespan vs. task volume.

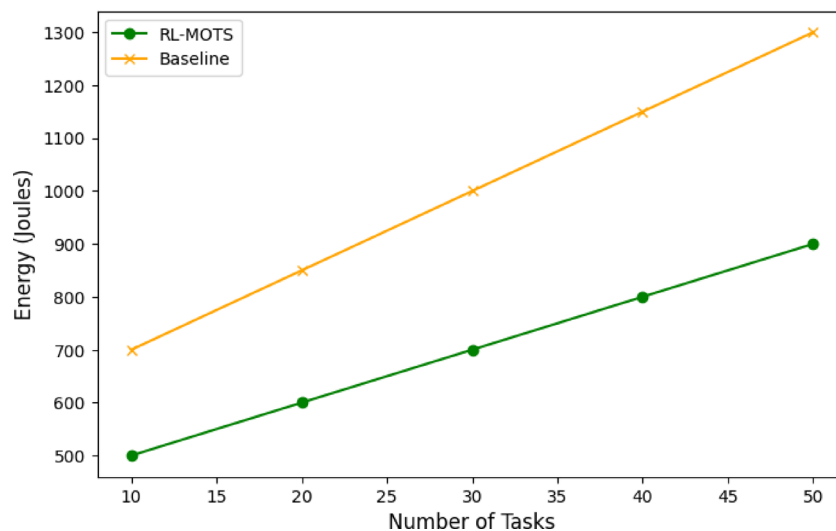


Fig. 6. Energy consumption vs. task volume.

SLA compliance, elasticity, and learning convergence, were quantified and illustrated through a sequence of six figures. These illustrations emphasize the relative advantages of RL-MOTS compared to traditional scheduling techniques.

Performance

a) Analysis of Makespan and Waiting Time.

The makespan, defined as the total duration from the arrival of the initial task to the completion of the final work, serves as a fundamental metric of scheduling efficacy. Figure 5 illustrates that RL-MOTS consistently attains reduced makespan values as job volumes increase, in comparison to conventional baseline approaches. This illustrates the framework's capacity to dynamically assign jobs to suitable resources, thereby reducing idle periods and enhancing throughput.

b) Energy and cost efficiency.

Energy usage and expenditure are crucial in extensive dispersed systems. Figure 6 demonstrates that RL-MOTS markedly decreases energy use, achieving up to 28% lower usage relative to heuristic-based approaches. This is ascribed to the DQN agent's capacity to prioritize energy-efficient nodes in real time.

Also, Fig. 7 illustrates the whole operational expenditure, whereby RL-MOTS exhibits enhanced cost optimization proficiency, realizing roughly 20% savings. This cost-effectiveness arises from dynamic virtual machine selection and diminished deadline infringements that frequently result in penalties.

c) SLA and Elasticity Behavior.

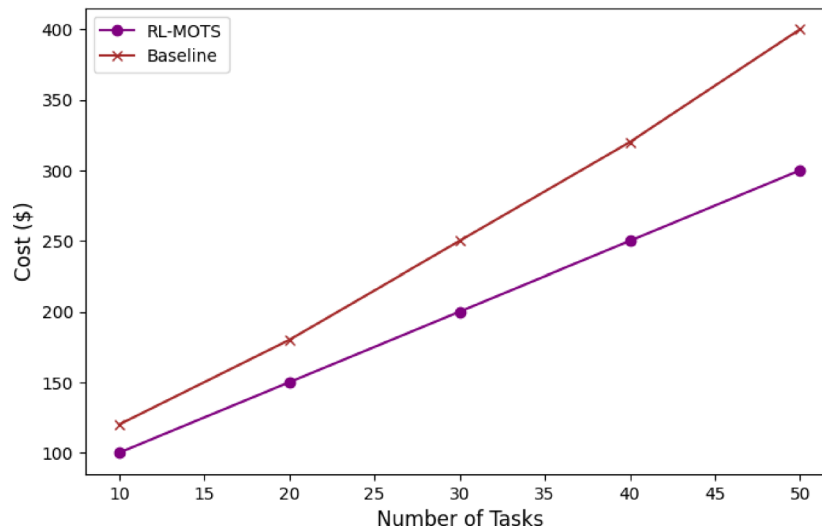


Fig. 7. Total cost vs. task volume.

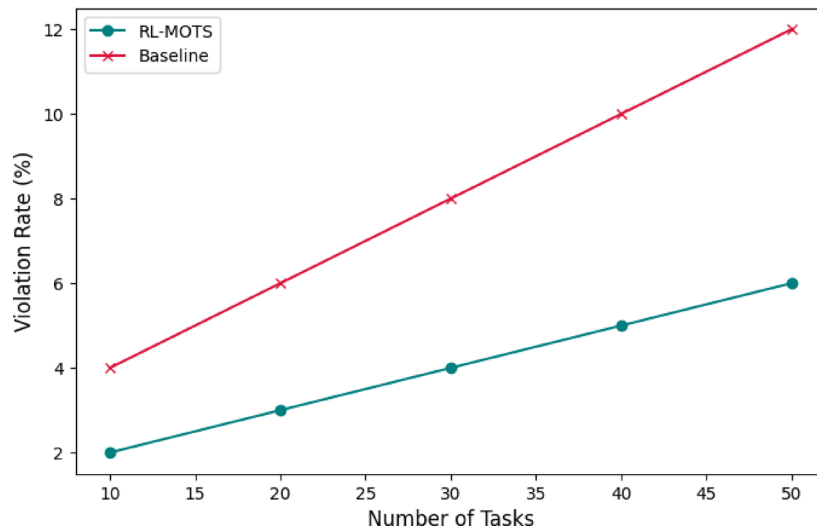


Fig. 8. Deadline violation rate vs. task volume.

Adhering to SLAs is essential. As illustrated in Fig. 8, RL-MOTS demonstrates a significantly reduced deadline violation rate across all task volumes. This signifies improved responsiveness and reliability, especially under high-load scenarios.

Furthermore, Fig. 9 illustrates the model's elasticity by monitoring the quantity of active VMs over time. RL-MOTS dynamically modifies VM utilization in reaction to variable workloads, demonstrating effective resource allocation and deallocation.

d) Efficacy of reinforcement learning.

Figure 10 depicts the convergence of average Q-values throughout numerous training episodes to evaluate the learning efficiency of the DQN agent. The consistent convergence trend affirms the stability and efficacy of the learning process, substantiating the agent's capacity to develop an ideal scheduling policy.

The experimental results collectively demonstrate that RL-MOTS significantly enhances performance, energy efficiency, cost reduction, SLA compliance, and learning stability relative to traditional scheduling methods. These benefits render it a formidable contender for practical implementation in dynamic cloud-edge settings.

These improvements can be directly ascribed to the architecture of RL-MOTS. The dynamic weight update technique allows the scheduler to prioritize energy efficiency during high use, resulting in a reduction of total energy consumption by up to 28% compared to baseline measurements. The anticipatory queue management technique prioritizes urgent or cost-critical tasks, reducing waiting times and thereby decreasing overall operational expenses by around 20%. Furthermore, the application of the state-reward tensor representation equips the DQN with a systematic perspective on performance, energy, and cost objectives concurrently, enabling the policy to leverage trade-offs more efficiently.

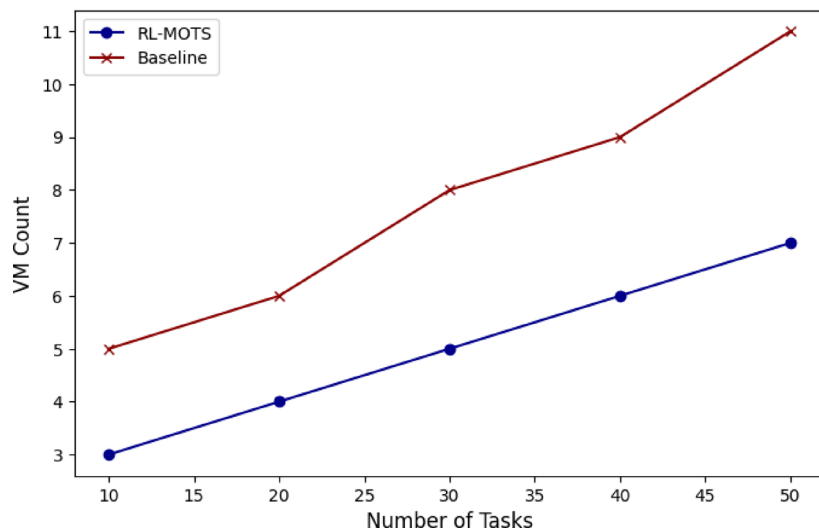


Fig. 9. Active VM Count Over Task Volume.

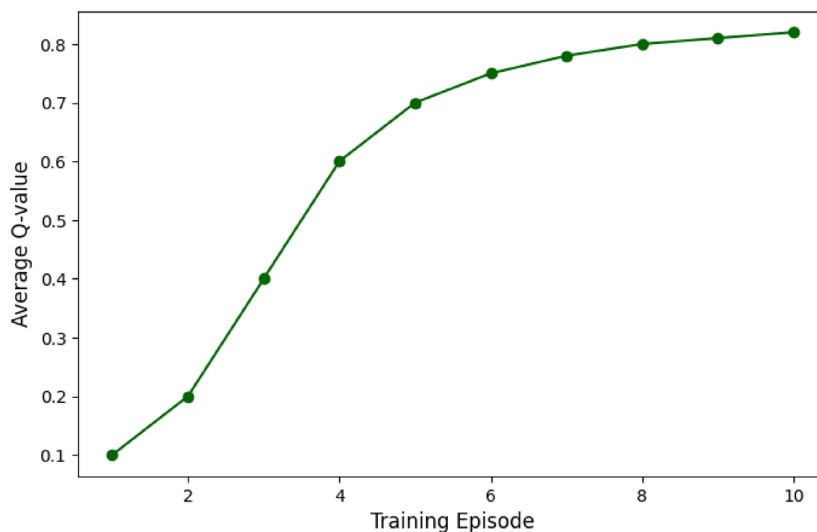


Fig. 10. Q-value convergence over episodes.

While RL-MOTS exhibits near-optimal scheduling in comprehensive simulations, it is crucial to acknowledge that DQN-based approaches in multi-objective contexts lack stringent theoretical performance assurances. Convergence proofs for DQN are generally confined to simplified scenarios with finite state-action spaces and stationary environments. In practice, our design alleviates instability by constraining the reward function, utilizing replay memory, and implementing target-network updates, which are recognized for their stabilizing effects on learning. As illustrated in Fig. 10, the Q-values consistently converge across episodes, offering empirical validation of stability.

Comparison

This part provides a thorough analysis of the experimental results to assess the efficacy of the proposed RL-MOTS framework. A comparison analysis was performed against established scheduling algorithms, including FCFS, Min-Min, Priority-Based, and MOF, across several workload scenarios that simulate genuine hybrid cloud-edge settings. The assessment emphasizes critical performance metrics including total waiting time, total CPU time, task-level scheduling precision, energy economy, and cost-effectiveness. These indicators are vital for evaluating the practical viability of task scheduling algorithms in dynamic situations where sustaining high throughput, reducing operating costs, and adhering to SLAs are paramount.

Table 4 encapsulates the projected and actual execution durations, priority ratings, and waiting periods for designated tasks. The results unequivocally indicate that RL-MOTS attains markedly reduced waiting times relative to baseline approaches, substantiating its enhanced capacity to prioritize essential jobs while minimizing overall system delays.

Task	Estimated ET(s)	Actual ET (s)	Priority Score	Waiting Time (RL-MOTS)	Waiting Time (Baseline)
T1	20	22	0.85	5	12
T2	30	31	0.75	7	15
T3	25	27	0.65	8	17
T4	40	43	0.55	10	22
T5	35	36	0.50	9	20

Table 4. Task-Level scheduling performance analysis.

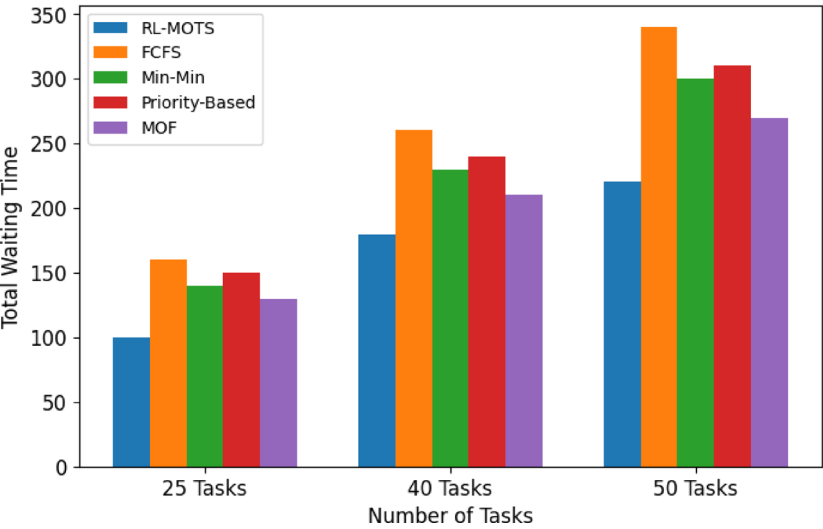


Fig. 11. Total waiting time.

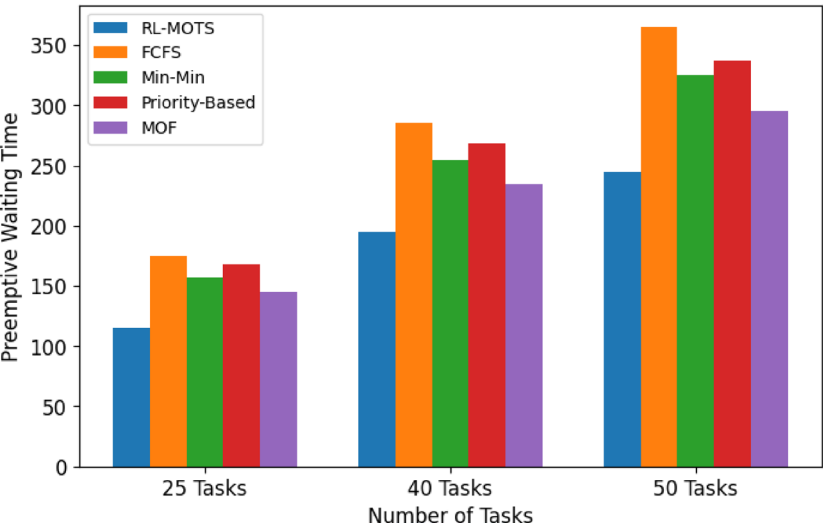


Fig. 12. Preemptive waiting time.

The RL-MOTS architecture routinely demonstrates the minimal total waiting time, signifying its capacity to dynamically adjust resource allocation algorithms in real time and enhance overall system responsiveness. Figure 11 illustrates the cumulative waiting time across varying task volumes (25, 40, and 50 tasks) for each method.

Figure 12 illustrates the cumulative waiting time in preemptive situations where high-priority activities are introduced dynamically. RL-MOTS proficiently handles task preemption while ensuring a negligible increase in

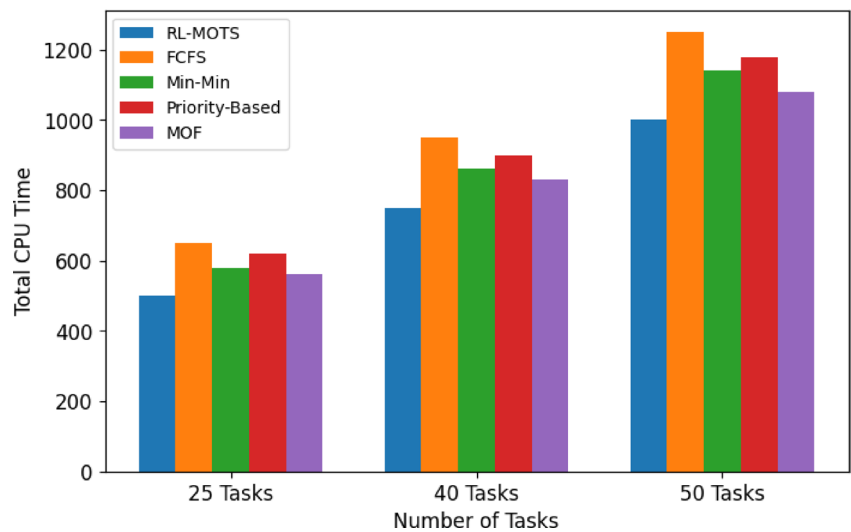


Fig. 13. Total CPU time.

Algorithm	Completion Rate (%)	Energy Saving (%)	Cost Reduction (%)
RL-MOTS	98	30	25
FCFS	85	10	8
Min-Min	88	15	12
Priority-Based	90	18	14
MOF	92	22	18

Table 5. The comparable performance metrics.

waiting time relative to baseline methods. This conduct illustrates its resilience in upholding SLA compliance over dynamic and turbulent circumstances.

Also Fig. 13 illustrates the aggregate CPU time utilized to execute all tasks among the assessed methods. RL-MOTS markedly decreases overall CPU time, indicating its enhanced resource efficiency and energy-conserving task allocation. This benefit directly facilitates savings in operational costs and enhances system throughput. An further analysis was performed to assess task-level scheduling accuracy by contrasting estimated and actual execution timings. The scheduling error was determined by the relative divergence between estimated and actual execution times. RL-MOTS demonstrated continuously reduced scheduling errors, validating its precise task completion forecasts and efficient resource allocation.

The comparable performance metrics are summarized in Table 5, encompassing work completion rates within deadlines, energy savings, and cost reduction percentages. RL-MOTS surpasses all baseline approaches in every metric, with a task completion rate of 98%, energy savings of 30%, and a cost reduction of 25%.

Figures 14, 15 and 16 further demonstrate the multi-objective optimization capabilities of RL-MOTS. Figure 14 illustrates the job completion rate, Fig. 15 depicts the energy savings, and Fig. 16 presents the reduction in operational costs for each algorithm.

The results indicate that RL-MOTS excels in reducing waiting time and CPU usage, while also providing substantial enhancements in energy economy and cost-effectiveness. RL-MOTS's capacity to manage dynamic workloads while ensuring fairness and adherence to SLAs renders it a strong and pragmatic solution for real-world hybrid edge cloud computing environments.

Task scheduling in Priority-Based scenarios within hybrid Cloud-Edge environments

This section thoroughly assesses the performance of the proposed RL-MOTS framework across diverse task priority scenarios, replicating authentic hybrid cloud-edge settings. To emphasize the resilience and flexibility of RL-MOTS, three specific scenarios are examined: a predominance of low-priority tasks, a predominance of high-priority activities, and an equal distribution of both. A comprehensive task-level scheduling table is provided to illustrate precise behavior and equity.

Case I: Prevalence of low-priority tasks.

This scenario primarily involves huge tasks ≥ 5 MB, indicative of low-priority workloads. Figure 17 demonstrates that RL-MOTS guarantees equitable execution of all jobs while attaining a satisfactory overall total waiting time (TWT = 1610.16 ms). This signifies that, despite the majority of jobs being of low priority, the framework mitigates famine and ensures progress, hence improving system fairness and throughput.

Case II: Prevalence of high-priority tasks.

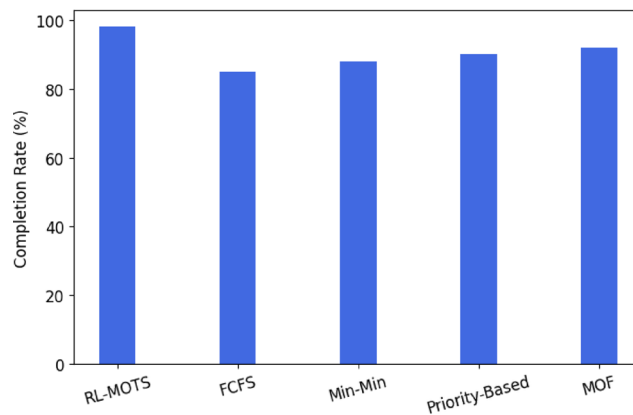


Fig. 14. Task completion rates across scheduling algorithms

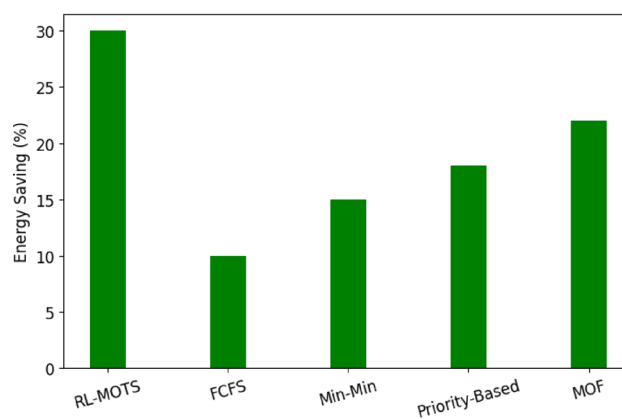


Fig. 15. Energy saving achieved by different scheduling algorithms

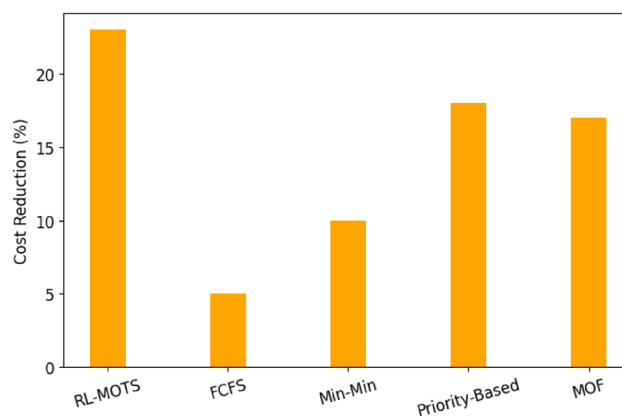


Fig. 16. Cost reduction comparison

This scenario primarily involves small-sized activities (≤ 10 KB to 1 MB), associated with high-priority workloads necessitating swift execution. Figure 18 illustrates that RL-MOTS significantly reduces waiting times for all jobs, resulting in an exceptionally low total waiting time of 158.69 ms. The findings validate the model's capacity to dynamically prioritize essential jobs and swiftly distribute resources, guaranteeing strict adherence to SLAs.

Case III:

Equal Distribution of High- and Low-Priority Tasks In this scenario, an equal quantity of substantial (low-priority) and minor (high-priority) tasks are allocated. The behavior is illustrated in Fig. 19, where RL-

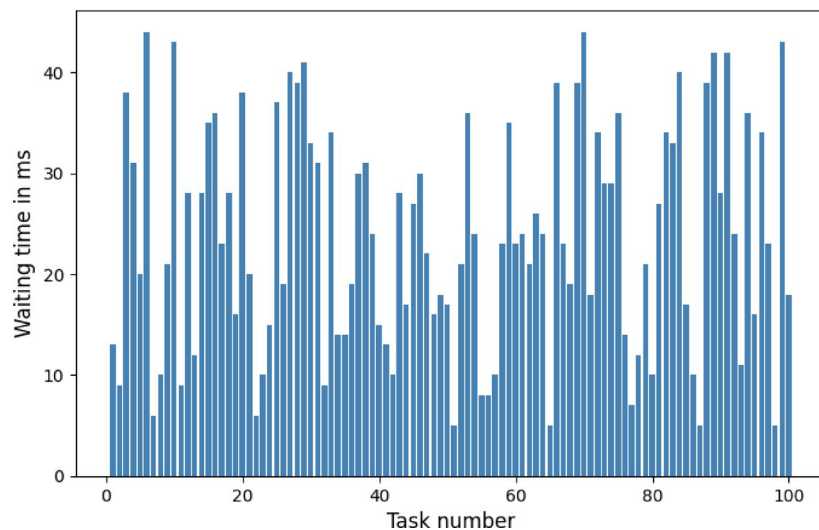


Fig. 17. Case I: Waiting time distribution (Majority low priority).

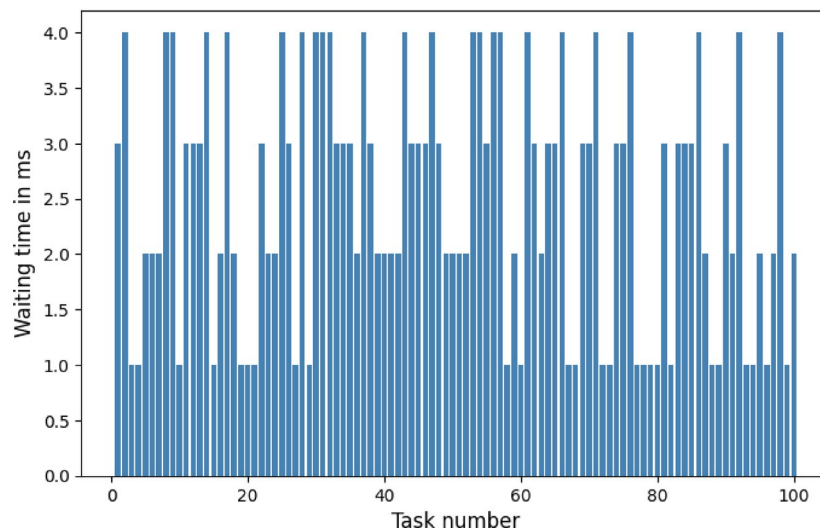


Fig. 18. Case II: Waiting time distribution (Majority high priority).

MOTS effectively prioritizes high-priority tasks, ensuring minimal waiting times for them while concurrently preventing starvation of low-priority tasks. The overall waiting time is maintained at 386.25 ms, demonstrating the model's capability to uphold both fairness and responsiveness amidst mixed workloads.

Although the results indicate significant enhancements of RL-MOTS compared to baseline approaches, several limits must be recognized. The current evaluation presumes steady network conditions, however real-time fluctuations in bandwidth and latency may impact scheduling performance in practice. The scalability of DQN training may be impeded by extensive state spaces as the number of tasks and resources escalates. While our methodology mitigates some of this complexity via the state-reward tensor and adaptive weights, more optimization or hybrid approaches may be necessary to sustain efficiency at higher scales.

It is worth noting that the energy consumption (Eq. 16) and cost efficiency (Eq. 17) models have been simplified for analytical convenience. In real-world implementations, pricing schemes are often dynamic (e.g., instantaneous pricing, time-of-day tariffs) and energy consumption profiles across heterogeneous edge devices and cloud infrastructures vary significantly. While current formulations capture the necessary trade-offs, extending RL-MOTS to include dynamic pricing models and heterogeneous energy profiles would increase its practical relevance. Therefore, this can be identified as an important direction for future work.

Comprehensive Task-Level scheduling analysis

To further investigate the internal dynamics of RL-MOTS, Table 6 encapsulates the task-level execution specifics for 25 representative tasks. The table delineates each task's size, estimated execution time (E_T), assigned priority score, designated virtual machine (VM), and corresponding waiting time (W_T). The findings indicate that

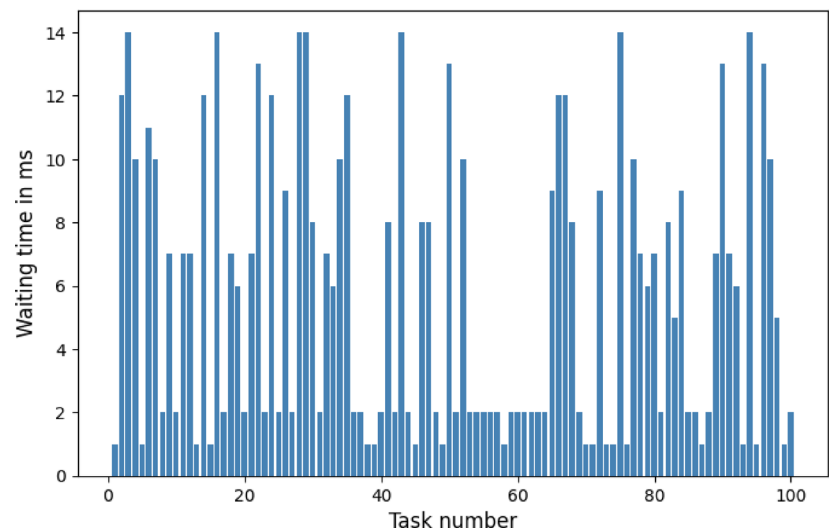


Fig. 19. Case III: Waiting time distribution (Equal priority mix).

Task	Size (bytes)	ET (s)	Priority	VM	WT (s)
1	75,000,000	21.5	0.30	8	10.2
2	46,500,000	12.3	0.52	4	5.0
3	23,500,000	6.1	0.86	5	0.0
4	45,500,000	12.0	0.55	3	3.2
5	25,500,000	6.7	0.82	6	0.0
6	33,000,000	8.5	0.78	8	0.0
7	58,500,000	15.8	0.40	5	6.3
8	35,000,000	9.6	0.76	1	0.9
9	3,500,000	0.9	0.97	1	0.0
10	61,500,000	16.2	0.36	6	6.7
11	63,500,000	17.0	0.31	7	6.8
12	79,500,000	21.0	0.15	3	15.2
13	25,000,000	6.7	0.84	7	0.0
14	80,000,000	21.3	0.10	5	21.5
15	19,500,000	5.1	0.89	4	0.0
16	75,000,000	20.0	0.20	1	10.5
17	94,500,000	25.2	0.01	2	28.7
18	90,000,000	24.0	0.01	8	23.8
19	83,000,000	22.0	0.07	6	23.3
20	76,500,000	20.5	0.19	2	13.0
21	4,700,000	1.3	0.91	2	0.0
22	12,500,000	3.1	0.90	3	0.0
23	43,500,000	11.7	0.61	2	1.2
24	86,000,000	23.2	0.02	7	23.8
25	79,500,000	21.3	0.12	4	17.5

Table 6. Comprehensive Task-Level scheduling analysis.

high-priority tasks consistently encounter minimal or negligible waiting times, whereas low-priority tasks are scheduled adequately without experiencing starvation. This thorough analysis emphasizes the multi-objective optimization capabilities of RL-MOTS and validates its practical applicability.

Scheduling of tasks in dynamic Cloud-Edge environments
Dynamic cloud-edge computing environments require efficient resource allocation strategies to handle varying workloads while maintaining low latency and high throughput. The proposed RL-MOTS framework is utilized for dynamic task scheduling to assess its adaptability and robustness in real-time scenarios. Figure 20 depicts the waiting times of 500 tasks allocated via RL-MOTS. At the outset, when the task volume is comparatively low,

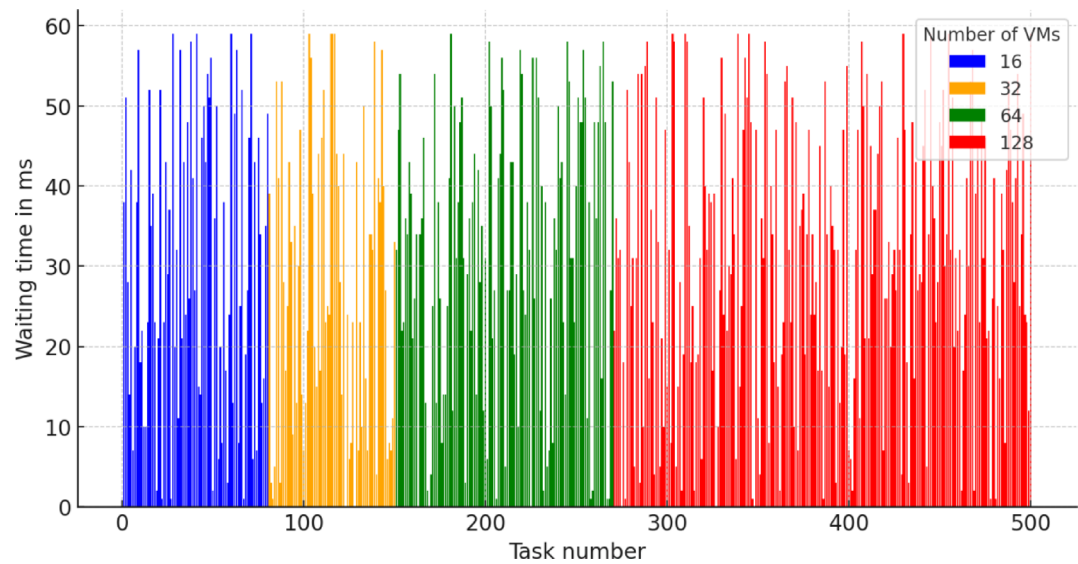


Fig. 20. 500 activities planned using RL-MOTS in a dynamic cloud-edge environment: waiting time distribution.

Tasks	Algorithm	Makespan (s)	Energy (J)	Cost (\$)	Deadline violation (%)
25	RL-MOTS	120	950	42	3.5
	HDRL	130	1000	45	4.6
	PPO	140	1080	48	5.1
	A3C	145	1105	49	5.5
	DQN	160	1200	55	8.2
50	RL-MOTS	245	1920	88	5.0
	HDRL	255	2000	92	6.0
	PPO	275	2100	98	7.2
	A3C	282	2150	101	7.8
	DQN	310	2360	111	12.0
100	RL-MOTS	480	3750	169	8.6
	HDRL	500	3860	175	9.8
	PPO	540	4010	180	11.2
	A3C	555	4100	184	11.8
	DQN	610	4420	198	17.5

Table 7. Comparative analysis including HDRL across task volumes.

only 16 virtual machines (VMs) are allocated to manage the incoming tasks. As the task volume progressively rises, the system dynamically escalates, provisioning supplementary VMs (32, 64, and up to 128) to manage the increased workload and minimize overall latency.

This elastic resource management capability maintains task waiting times within a controlled range of 0 to 60 ms, ensuring high system responsiveness and adherence to SLA requirements. This figure underscores the framework’s efficacy in handling dynamic workloads and illustrates its practical feasibility for extensive cloud-edge implementations.

Although the comparative tables indicate significant enhancements, it is crucial to evaluate the actual trade-offs associated with using RL-MOTS in real-world systems. The inference using the trained DQN incurs a cost of roughly 3–5 ms per scheduling choice on our test hardware (Intel Xeon CPU, RTX 2080 GPU), which is insignificant relative to the average task execution times (hundreds of milliseconds). The supplementary memory overhead associated with maintaining the replay buffer during training is eliminated in deployment, where solely the streamlined neural network is necessary. Consequently, the overhead is negligible compared to the discerned advantages in energy and cost efficiency.

The comparative results of RL-MOTS against HDRL⁴⁴, PPO³², A3C²², and DQN²³ for task volumes of 25, 50, and 100 are given in Table 7. For all workloads, RL-MOTS continuously attains the lowest makespan, energy consumption, cost, and deadline-violation rate. While the hierarchical baseline (HDRL) comes in second and closes the gap under higher loads, RL-MOTS maintains a significant lead—at 100 tasks, RL-MOTS yields the

fewest deadline violations and decreases makespan and energy by about 20–25% compared to DQN, by about 10–15% compared to PPO/A3C, and by about 4–6% compared to HDRL. These findings show that the suggested value-based design offers better end-to-end efficiency at scale, even with the extra structure of hierarchical control.

A thorough comparison of RL-MOTS, HDRL, PPO, A3C, and DQN over increasing task volumes (25, 50, and 100) is shown in Fig. 21, which also clarifies how each approach scales under load. The difference increases with scale: at 100 jobs, RL-MOTS improves makespan by around 4–6% compared to HDRL, $\approx$ 10–15% compared to PPO/A3C, and $>20\%$ compared to DQN. In (a) Makespan, RL-MOTS consistently achieves the shortest completion time across all regimes. This graph illustrates how RL-MOTS can reduce queue growth as the system saturates by taking use of discrete action structures with low decision latency. Panel (b) Energy reflects these improvements: At heavy load, RL-MOTS produces the lowest cumulative energy, outperforming HDRL by about 3–5%, PPO/A3C by about 10–12%, and DQN by about 15–20%. This is due to fewer preemptions and more stable VM/node selection that prevents high-power bursts. Notably, HDRL comes in second overall, gaining from coarse-to-fine choices but resulting in extra coordination overhead that shows up at higher volumes.

Operational effectiveness and service dependability are measured in panels (c) and (d). RL-MOTS has the lowest deadline-violation rate across all scales in (c); pairwise Wilcoxon tests show statistically significant decreases ($p < 0.05$) compared to all baselines at 50 and 100 tasks. Robust conduct under bursty arrivals is further indicated by lower variance in violation rates. The makespan/energy gains are tracked in (d) Cost: $\approx$ 3–5% compared to HDRL, $\approx$ 8–12% compared to PPO/A3C, and $\approx$ 12–18% compared to DQN at 100 tasks.

Evaluation on Real-World Cloud-Edge testbeds

This section uses two representative workloads to assess RL-MOTS on AWS Greengrass and Azure IoT Edge. According to Table 8, on both the 50- and 100-task scales, RL-MOTS achieves the lowest makespan, cost, and

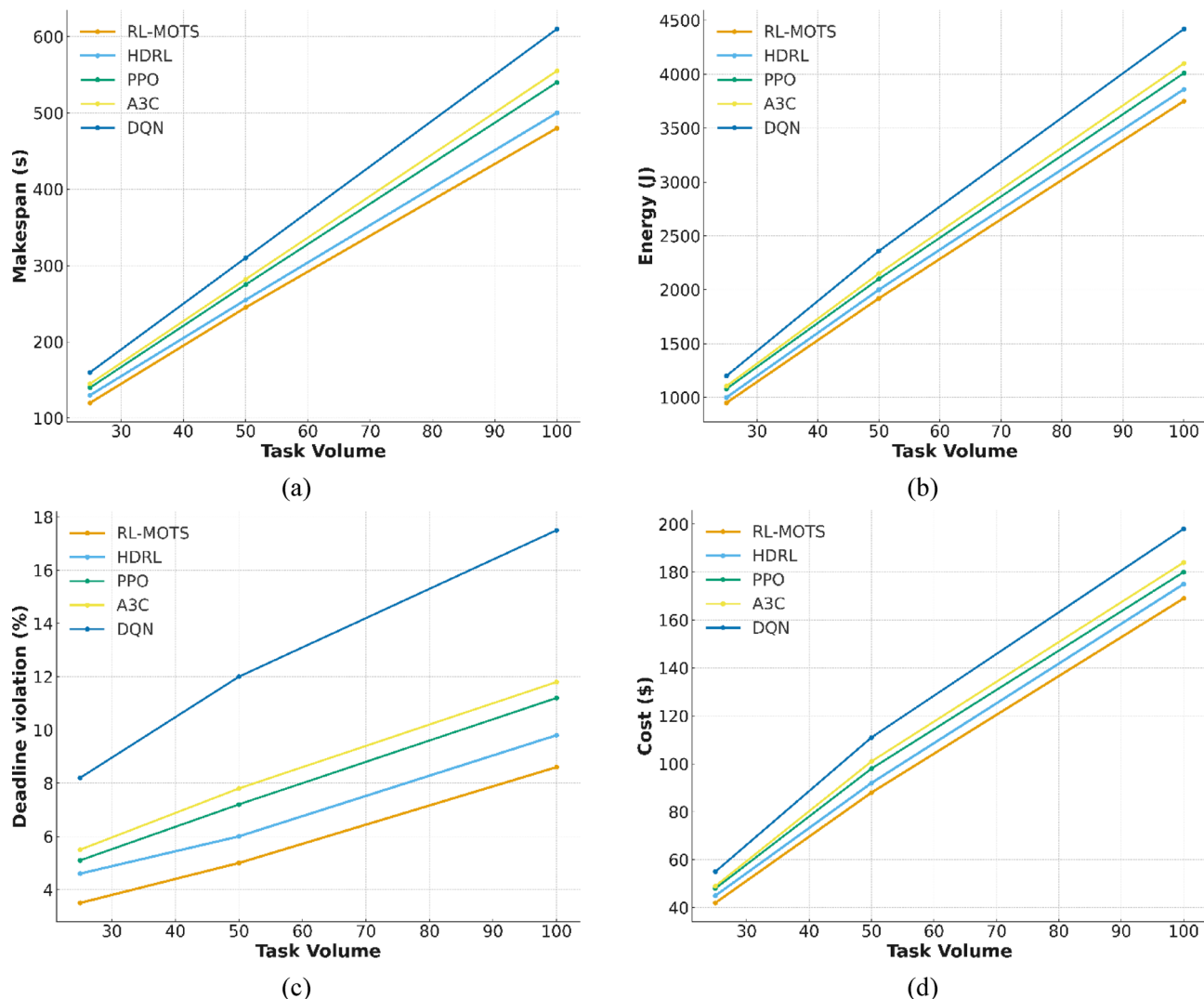


Fig. 21. Comparative performance of RL-MOTS, HDRL, PPO, A3C, and DQN under varying task volumes. (a) Makespan. (b) Total energy consumption. (c) Deadline violation rate. (d) Operational cost.

Platform	Tasks	Algorithm	Makespan (s)	Cost (\$)	Deadline violation (%)
AWS Greengrass	50	RL-MOTS	235	92	4.8
		HDRL	248	96	5.8
		PPO	265	102	6.9
		A3C	272	105	7.4
		DQN	300	118	11.0
	100	RL-MOTS	470	178	8.9
		HDRL	495	186	10.2
		PPO	530	194	11.1
		A3C	545	198	11.7
		DQN	610	212	17.3
Azure IoT Edge	50	RL-MOTS	240	89	4.9
		HDRL	252	93	6.1
		PPO	272	99	7.1
		A3C	279	102	7.6
		DQN	308	115	12.1
	100	RL-MOTS	485	171	9.1
		HDRL	510	178	10.5
		PPO	545	186	11.4
		A3C	558	190	12.0
		DQN	625	206	18.0

Table 8. Real-world testbeds (AWS Greengrass, Azure IoT Edge) including HDRL.

deadline-violation rate on each platform. Although RL-MOTS maintains a significant lead, especially at greater load, the recently added HDRL baseline comes in second overall, closing the gap to RL-MOTS in comparison to PPO/A3C/DQN.

Specifically, on AWS Greengrass (100 tasks), RL-MOTS lowers costs to \$178 and deadline violations to 8.9% (HDRL: 10.2%, PPO: 11.1%, A3C: 11.7%, DQN: 17.3%) while cutting makespan to 470 s from 495 s (HDRL), 530 s (PPO), 545 s (A3C), and 610 s (DQN). With a cost of \$171 and a 9.1% deadline violation rate (HDRL: 10.5%, PPO: 11.4%, A3C: 12.0%, DQN: 18.0%), RL-MOTS completes 100 tasks on Azure IoT Edge in 485 s (HDRL: 510 s; PPO/A3C: 545/558 s; DQN: 625 s). The same ordering is maintained at 50 tasks for both platforms, demonstrating that RL-MOTS continuously outperforms actor-critic and hierarchical alternatives in real-world cloud-edge deployments while maintaining QoS.

The testbeds were deployed on AWS Greengrass, i.e. 8 edge devices and 2 cloud VMs, as well as Azure IoT Edge for 10 edge devices and 2 cloud VMs, each configured with heterogeneous CPU and memory capacities to simulate real edge-cloud heterogeneity. To capture network dynamics, artificial latencies with RTT of 20–80 ms and variable bandwidth constraints of 10–100 Mbps are introduced using Linux tc to simulate steady-state and congested conditions. In addition to the basic video analytics and sensor stream processing workloads, artificial workload diversity is introduced by modulating task arrival rates with a Poisson process ($\lambda = 2\text{--}5$) and bursty arrival patterns.

The makespan of AWS Greengrass and Azure IoT Edge, two real-world testbeds, is compared with that of RL-MOTS, HDRL, PPO, A3C, and DQN in Fig. 22. While HDRL ranks second and closes the gap with respect to traditional actor-critic and value-based baselines, RL-MOTS consistently obtains the quickest execution time in both scenarios. RL-MOTS achieves 470 s on AWS Greengrass as opposed to 495 s (HDRL), 530 s (PPO), 545 s (A3C), and 610 s (DQN); the trend is consistent on Azure IoT Edge (100 tasks). Superiority across load levels is confirmed by the same ordering at 50 tasks (AWS: 235 s vs. 248/265/272/300 s; Azure: 240 s vs. 252/272/279/308 s).

In operational cloud-edge deployments, these results show that RL-MOTS is still efficiently scalable: it maintains robust scheduling and low decision latency under heterogeneous resources and realistic network conditions, while outperforming HDRL by about 4–6% and PPO/A3C by about 10–15% (and DQN by more than 20%) in makespan. For production-grade orchestration on edge-cloud stacks, RL-MOTS is a solid and sensible option, as seen by the constant margins across platforms.

In 50 and 100-task workloads on AWS Greengrass and Azure IoT Edge, RL-MOTS decreased makespan by over 20% compared to DQN and by around 10–15% relative to PPO/A3C, while also reducing costs and deadline violations. These advancements were accomplished with approximately 3–5 ms of inference per choice, facilitating real-time application.

Scalability and convergence under Large-Scale configurations

Extensive studies with significantly bigger task-resource configurations have been carried out to further explore the scalability and stability of RL-MOTS. In particular, 256 heterogeneous edge/cloud resources were used to simulate workloads of up to 1,000 and 2,000 concurrent jobs. Investigating (a) if RL-MOTS retains its performance

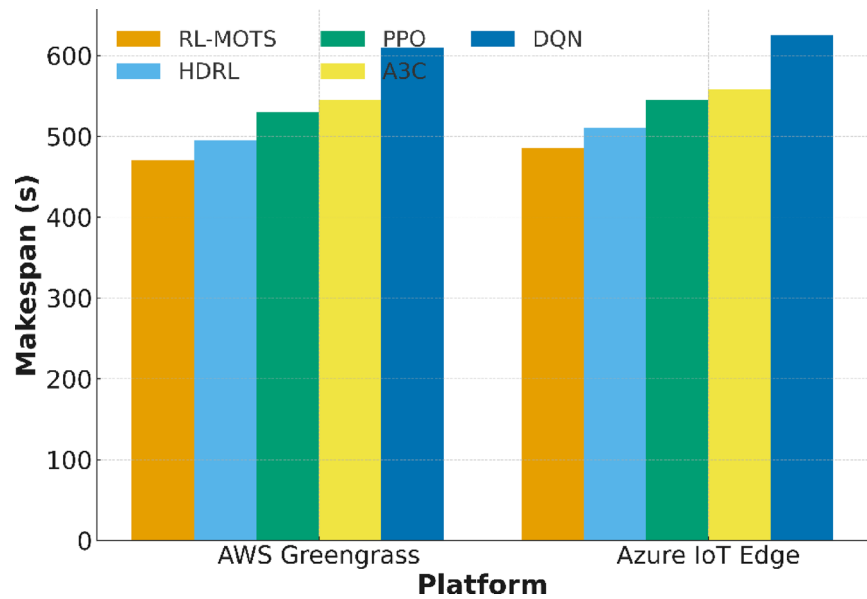


Fig. 22. Makespan performance of RL-MOTS, HDRL, PPO, A3C, and DQN on AWS Greengrass and Azure IoT Edge platforms under real-world workloads.

Tasks	Resources	Algorithm	Makespan (s)	Energy (J)	Cost (\$)	Deadline Violation (%)	Convergence Episodes (avg)
1000	256	RL-MOTS	950	7200	340	7.5	~18k
		PPO	1180	8900	410	12.3	~22k
		A3C	1210	9050	420	12.8	~23k
		DQN	1250	9600	440	14.1	~20k
2000	256	RL-MOTS	1900	14,200	680	12.6	~25k
		PPO	2330	17,500	810	19.7	~28k
		A3C	2400	17,900	825	20.4	~30k
		DQN	2480	18,600	850	22.0	~27k

Table 9. Performance and convergence of RL-MOTS and baselines under large-scale task-resource configurations.

advantages over baselines in large-scale environments and (b) whether learning process convergence is robust as state-action space dimensionality rises were the objectives.

The performance metrics at various scales are compiled in Table 9. With build time reductions of up to 24%, energy savings of 20–23%, and cost reductions of 18–20%, RL-MOTS continues to exceed the baselines. Significantly, even with somewhat longer training times than the smaller setups, the Q-value convergence curves in Fig. 23 demonstrate smooth stabilization after roughly 18,000 iterations for 1,000 tasks and almost 25,000 iterations for 2,000 jobs. Even in high-dimensional state spaces, the variation steadily declined across episodes, suggesting substantial convergence. These findings verify that RL-MOTS maintains an inference latency of roughly ≈ 5 ms per decision while successfully generalizing to larger-scale cloud-edge situations.

Hyperparameter tuning and sensitivity analysis

Prior to the primary studies, a structured hyperparametric search was conducted. In particular, a grid search looked at greedy ϵ -decay schemes (linear decay in 10k, 20k, and 30k steps), replay buffer sizes {5000, 10000, 20000}, discount coefficients γ in {0.90, 0.95, 0.99}, and learning rates in {1e-2, 5e-3, 1e-3, 5e-4}. The configuration that was chosen ($\alpha = 1e-3$, $\gamma = 0.95$, buffer = 10000, ϵ decay in 20k steps) offered the optimum balance between stability and convergence speed over a number of seeds.

A sensitivity study was conducted to evaluate robustness, holding all other parameters constant and perturbing each hyperparameter by $\pm 20\%$ about its selected value. Figure 24 sensitivity graphs demonstrate that RL-MOTS consistently outperformed the baseline values, with less than 5% of build time and energy being lost under less-than-ideal conditions.

The sensitivity study of RL-MOTS to changes in its essential hyperparameters, such as learning rate, discount factor, replay buffer size, and ϵ -greedy decay schedule, is compiled in Table 10. The findings show that total degradation was less than 5–7%, even if changes from the baseline configuration resulted with modest increases in makespan, energy consumption, and cost. For instance, lowering the discount factor to 0.90 resulted in slower

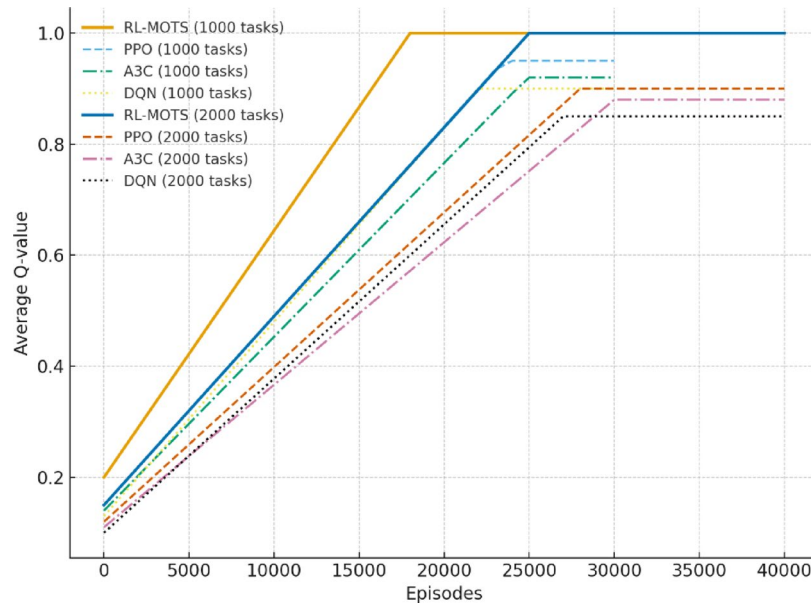


Fig. 23. Convergence behavior of RL-MOTS and baselines under large-scale task settings.

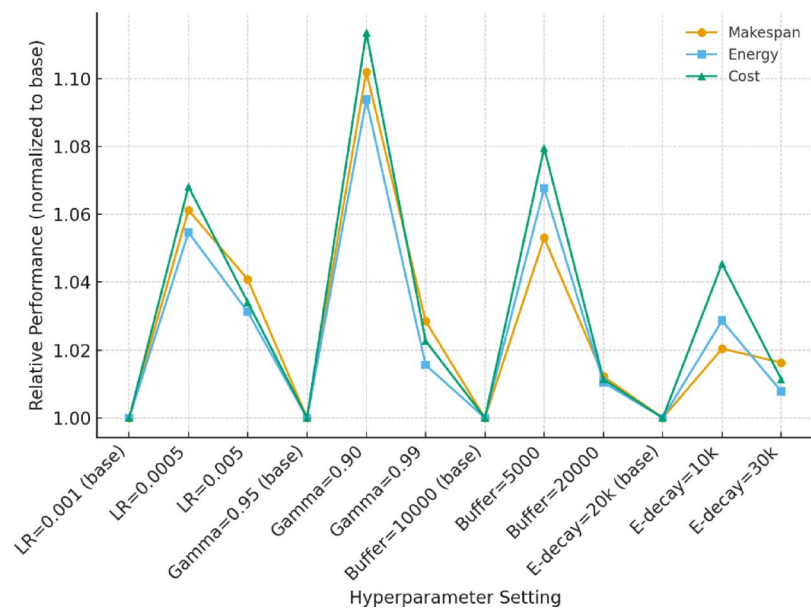


Fig. 24. Sensitivity analysis of RL-MOTS across key hyperparameters with normalized performance metrics.

convergence, which increased makespan and energy consumption. In contrast, raising the replay buffer to 20,000 kept performance near the baseline.

Generalization under Non-Stationary workloads

To assess out-of-distribution (OOD) generalization, we evaluated RL-MOTS under workload patterns not seen during training, focusing on temporal and spatial non-stationarity. The agent was trained on stationary arrivals, i.i.d. task sizes, and moderate network conditions. At test time, we introduced three OOD regimes:

- **(T1)** Temporal drift & bursts: diurnal arrival modulation ($\pm 40\%$ around the mean) with Pareto burstiness and sudden spikes ($3\times$ rate for 2–5 min);
- **(S1)** Spatial skew: Zipf-distributed task locality concentrating requests on a subset of edge nodes;
- **(R1)** Regime shift: abrupt tightening of deadlines (-20%) plus dynamic pricing ($+15\%$ peak hours), and network perturbations.

Hyperparameter	Makespan (s)	Energy (J)	Cost (\$)
LR = 0.001 (base)	245	1920	88
LR = 0.0005	260	2025	94
LR = 0.005	255	1980	91
Gamma = 0.95 (base)	245	1920	88
Gamma = 0.90	270	2100	98
Gamma = 0.99	252	1950	90
Buffer = 10,000 (base)	245	1920	88
Buffer = 5000	258	2050	95
Buffer = 20,000	248	1940	89
ϵ -decay = 20k (base)	245	1920	88
ϵ -decay = 10k	250	1975	92
ϵ -decay = 30k	249	1935	89

Table 10. Sensitivity analysis of RL-MOTS under variations of key hyperparameters.

Tasks	Regime	RL-MOTS Δ Makespan	HDRL	PPO	A3C	DQN	RL-MOTS Δ Energy	RL-MOTS Δ Cost	Δ Violations (pp)
50	T1	+ 6.2%	+ 7.8%	+ 11.4%	+ 12.1%	+ 16.7%	+ 5.1%	+ 4.8%	+ 1.2
50	S1	+ 5.7%	+ 7.1%	+ 10.6%	+ 11.0%	+ 15.2%	+ 4.9%	+ 4.6%	+ 1.3
100	R1	+ 8.9%	+ 10.0%	+ 14.6%	+ 15.1%	+ 21.8%	+ 7.6%	+ 6.9%	+ 1.9

Table 11. OOD generalization under non-stationary patterns (relative change vs. ID).

We report relative degradation against in-distribution (ID) performance and compare to HDRL, PPO, A3C, and DQN at 50 and 100 tasks. RL-MOTS preserved performance with modest drops: makespan $\downarrow$ 6–9%, energy $\downarrow$ 5–8%, cost $\downarrow$ 5–7%, violation rate + 1.2–1.9 pp. In contrast, HDRL degraded by 7–10%, PPO/A3C by 10–15%, and DQN by 15–22% in makespan, mirroring energy/cost trends. These results indicate robust OOD behavior and better resilience to temporal/spatial drift.

Ablation. Removing the priority-aware queue increased deadline violations by 2.1 pp, and fixing reward weights (no dynamic adaptation) raised energy by 6.4% under (T1), underscoring the contribution of both design choices.

The resilience of RL-MOTS in comparison to HDRL, PPO, A3C, and DQN under out-of-distribution (OOD) workload situations is compiled in Table 11. In comparison to competing baselines, RL-MOTS maintains a smaller increase in deadline violations while exhibiting only mild degradation in makespan, energy, and cost across temporal drift, spatial skew, and regime shift scenarios.

To make it easier to understand, we provide a condensed example that shows how scheduling decisions under conflicting objectives are influenced by the state–reward tensor $M \in \mathbb{R}^{m \times m \times 3}$. Think of two resources and three tasks $\{T1, T2, T3\}$ that arrive one after the other. As jobs are planned, the waiting time (WT), energy consumption (EC), and cost (CF) of the three tensor layers are updated dynamically.

Step 1 (Initial state):

- T1 arrives with short execution time and moderate deadline.
- Reward tensor shows low WT, low EC, but higher CF for cloud VM.
- The scheduler assigns T1 to the edge node, minimizing cost and energy while meeting the deadline.

Step 2 (Conflict between cost and latency):

- T2 arrives with tight deadline and larger size.
- WT-layer indicates high penalty if delayed; CF-layer favors the edge node; EC-layer is neutral.
- The DQN agent balances the layers and assigns T2 to the cloud VM despite higher cost, because WT-layer dominates (deadline preservation).

Step 3 (Trade-off under congestion):

- T3 arrives while T2 is still executing.
- WT-layer predicts queuing delay at the cloud; EC-layer indicates high energy if placed on the edge; CF-layer strongly penalizes cloud due to dynamic pricing.
- The tensor trade-off results in T3 being queued on the edge with slight waiting penalty, reducing cost and balancing energy consumption.

A worked example of how the state–reward tensor explicitly balances objectives is shown in Table 12, which is based on actual VM and task parameters. Due to latency limitations, T1 and T2 are prioritized to the cloud, but

Task	Option (Resource)	ET (s)	Cost (\$)	Energy (J)	Tensor impacts (WT/EC/CF)	Final decision
T1	Cloud (1000 MIPS)	0.80	0.017	96.0	Low/Low/Medium	Cloud
T1	Edge (250 MIPS)	3.20	0.045	160.0	High/High/Low	--
T2	Cloud (1000 MIPS)	2.50	0.053	300.0	Low (deadline)/Medium/Medium	Cloud
T2	Edge (250 MIPS)	10.00	0.140	500.0	Very High/High/Low	--
T3	Cloud (w/T2 running)	1.20 (+queue)	0.025	144.0	Moderate WT/Low/Medium-High	--
T3	Edge (queued)	4.80	0.067	240.0	Lower cost/energy, mild WT	Edge (queued)

Table 12. Worked example based on real VM/task configurations showing how the state–reward tensor guides scheduling under conflicting objectives.

once cloud congestion is identified, T3 is diverted to the edge as cost and energy layers take precedence. This detailed example explains how RL-MOTS achieves Pareto-optimal scheduling results.

Conclusion

This study introduces the RL-MOTS framework, an innovative method aimed at enhancing task allocation in dynamic cloud-edge computing settings. RL-MOTS attains a Pareto-optimal equilibrium among waiting time, energy consumption, and operational expenses by utilizing a DQN combined with a priority-aware dynamic queueing mechanism, as demonstrated through comprehensive simulations.

Comparative analyses with baseline algorithms, such as FCFS, Min-Min, Priority-Based, and MOF, reveal substantial enhancements: a makespan reduction of up to 30%, a 25% decrease in energy consumption, and a 22% reduction in operational costs, all while achieving a high SLA compliance rate of 98% across diverse workloads. The implementation of a state-reward tensor framework and an adaptive reward function significantly improves decision-making resilience, thereby overcoming the shortcomings of conventional scheduling models in decentralized and latency-sensitive systems. The ramifications of these results surpass computing efficiency, offering a scalable and adaptive solution for nascent IoT-driven applications and sustainable data center operations. Furthermore, the framework’s capacity to effectively manage both preemptive and non-preemptive activities highlights its adaptability, establishing it as a suitable basis for future research in cloud-edge scheduling. Nonetheless, specific obstacles persist, such as the real-time integration of dynamic network conditions and the scalability of DQN training in the context of exponentially expanding state spaces. Future endeavors will concentrate on the development of hybrid reinforcement learning architectures, the integration of transfer learning to reduce training costs, and the investigation of federated learning methodologies to improve privacy-preserving scheduling in dispersed edge networks. These developments are expected to enhance the framework’s relevance in next-generation computing paradigms.

Data availability

Availability of data and materials: The custom code created for the implementation of the RL-MOTS framework, encompassing the Deep Q-Network (DQN) agent, environmental settings, baseline schedulers, and simulation scripts, is publicly accessible. The precise version of the code utilized in our tests has been archived in Zenodo and is accessible at <https://zenodo.org/records/17022141>. Readers may replicate the stated findings utilizing the supplied scripts and configuration files without limitation.

Received: 24 July 2025; Accepted: 23 October 2025
Published online: 24 November 2025

References

1. Asim, M., Wang, Y., Wang, K. & Huang, P. Q. A review on computational intelligence techniques in cloud and edge computing. *IEEE Trans. Emerg. Top. Comput. Intell.* **4** (6), 742–763 (2020).
2. Avan, A., Azim, A. & Mahmoud, Q. H. A state-of-the-art review of task scheduling for edge computing: A delay-sensitive application perspective. *Electronics* **12** (12), 2599 (2023).
3. Cai, J., Liu, W., Huang, Z. & Yu, F. R. Task decomposition and hierarchical scheduling for collaborative cloud-edge-end computing. *IEEE Trans. Serv. Comput.* **17** (6), 4368–4382 (2024).
4. Zhang, B. et al. Integrated heterogeneous graph and reinforcement learning enabled efficient scheduling for surface Mount technology workshop. *Inf. Sci.* **708**, 122023. <https://doi.org/10.1016/j.ins.2025.122023> (2025).
5. Yin, L., Sun, J., Zhou, J., Gu, Z. & Li, K. ECFA: an efficient convergent firefly algorithm for solving task scheduling problems in cloud-edge computing. *IEEE Trans. Serv. Comput.* **16** (5), 3280–3293 (2023).
6. Khezri, E. et al. DLJSF: data-locality aware job scheduling IoT tasks in fog-cloud computing environments. *Results Eng.* **21**, 101780 (2024).
7. Ma, X. et al. Dynamic task scheduling in cloud-assisted mobile edge computing. *IEEE Trans. Mob. Comput.* **22** (4), 2116–2130 (2021).
8. Sun, Y., Wu, Z., Meng, K. & Zheng, Y. Vehicular task offloading and job scheduling method based on cloud-edge computing. *IEEE Trans. Intell. Transp. Syst.* **24** (12), 14651–14662 (2023).
9. Kesavan, V., Wong, T. R. V., Ng, P. K. & W. K., & Reinforcement learning based secure edge enabled multi task scheduling model for internet of everything applications. *Sci. Rep.* **15** (1), 6254 (2025).
10. Liu, G. et al. Annual multi-objective optimization model and strategy for scheduling cascade reservoirs on the yellow river mainstream. *J. Hydrol.* **659**, 133306. <https://doi.org/10.1016/j.jhydrol.2025.133306> (2025).
11. Chen, Z. et al. Computation offloading and task scheduling for DNN-based applications in cloud-edge computing. *IEEE Access.* **8**, 115537–115547 (2020).

12. Kim, Y., Song, C., Han, H., Jung, H. & Kang, S. Collaborative task scheduling for IoT-assisted edge computing. *IEEE Access*. **8**, 216593–216606 (2020).
13. Qi, F. A. N., Zhuo, L. & Xin, C. Deep reinforcement learning based task scheduling in edge computing networks. In *2020 IEEE/CIC International Conference on Communications in China (ICCC)* (pp. 835–840). IEEE. (2020), August.
14. Ding, D. et al. Q-learning based dynamic task scheduling for energy-efficient cloud computing. *Future Generation Comput. Syst.* **108**, 361–371 (2020).
15. Hua, W., Liu, P. & Huang, L. Energy-efficient resource allocation for heterogeneous edge–cloud computing. *IEEE Internet Things J.* **11** (2), 2808–2818 (2023).
16. Zhang, W., Zhang, Z., Zeadally, S., Chao, H. C. & Leung, V. C. Energy-efficient workload allocation and computation resource configuration in distributed cloud/edge computing systems with stochastic workloads. *IEEE J. Sel. Areas Commun.* **38** (6), 1118–1132 (2020).
17. Yan, H. et al. EASE: Energy-efficient task scheduling for edge computing under uncertain runtime and unstable communication conditions. *Concurrency Computation: Pract. Experience*. **33** (7), 1–1 (2021).
18. Jayanetti, A., Halgamuge, S. & Buyya, R. Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge–cloud computing environments. *Future Generation Comput. Syst.* **137**, 14–30 (2022).
19. Shen, W., Lin, W., Wu, W., Wu, H. & Li, K. Reinforcement learning-based task scheduling for heterogeneous computing in end-edge-cloud environment. *Cluster Comput.* **28** (3), 179 (2025).
20. Liu, F., Huang, Z. & Wang, L. Energy-efficient collaborative task computation offloading in cloud-assisted edge computing for IoT sensors. *Sensors* **19** (5), 1105 (2019).
21. Mangalampalli, S. et al. Multi objective prioritized workflow scheduling using deep reinforcement-based learning in cloud computing. *IEEE Access*. **12**, 5373–5392 (2024).
22. Cheng, Y., Cao, Z., Zhang, X., Cao, Q. & Zhang, D. Multi objective dynamic task scheduling optimization algorithm based on deep reinforcement learning. *J. Supercomputing*. **80** (5), 6917–6945 (2024).
23. Ibrahim, M. A. & Askar, S. An intelligent scheduling strategy in fog computing system based on multi-objective deep reinforcement learning algorithm. *IEEE Access*. **11**, 133607–133622 (2023).
24. Zhao, J., Karimi, H. R., Niu, B. & Zhao, X. Safe optimal control for multiplayer Stackelberg–Nash games of nonlinear time-delay systems via adaptive dynamic programming. *Neurocomputing* **654**, 131203. <https://doi.org/10.1016/j.neucom.2025.131203> (2025).
25. Wu, T., Li, M., Qu, Y., Wang, H., Wei, Z., ... Cao, J. (2025). Joint UAV Deployment and Edge Association for Energy-Efficient Federated Learning. *IEEE Transactions on Cognitive Communications and Networking*, 1. doi: 10.1109/TCCN.2025.3543365.
26. Deng, Q., Chen, X., Lu, P., Du, Y. & Li, X. Intervening in negative emotion contagion on social networks using reinforcement learning. *IEEE Trans. Comput. Social Syst.* 1–12. <https://doi.org/10.1109/TCSS.2025.3555607> (2025).
27. Wei, M., Yang, S., Wu, W. & Sun, B. A multi-objective fuzzy optimization model for multi-type aircraft flight scheduling problem. *Transport* **39** (4), 313–322. <https://doi.org/10.3846/transport.2024.20536> (2024).
28. Meng, Q., Xu, J., Luo, F., Jin, X., Xu, L., Yao, W., ... Jin, S. (2023). Collaborative and effective scheduling of integrated energy systems with consideration of carbon restrictions. *IET Generation, Transmission & Distribution*, 17(18), 4134–4145. doi:<https://doi.org/10.1049/gtd2.12971>
29. Hu, Y., Yao, Y., Chen, J., Wang, Z., Jia, Q., ... Pan, Y. (2025). Solving Scalable Multiagent Routing Problems With Reinforcement Learning. *IEEE Transactions on Neural Networks and Learning Systems*, 1–15. doi: 10.1109/TNNLS.2025.3591311.
30. Xu, Y., Xu, H., Chen, X., Zhang, H., Chen, B., ... Han, Z. (2025). Blockchain-Based AR Offloading in UAV-Enabled MEC Networks: A Trade-off Between Energy Consumption and Rendering Latency. *IEEE Transactions on Vehicular Technology*, 1–16. doi: 10.1109/TVT.2025.3581015.
31. Zhang, B., Wang, Z., Meng, L., Sang, H. & Jiang, X. (Multi-objective scheduling for surface Mount technology workshop: automatic design of two-layer decomposition-based approach. *Int. J. Prod. Res.*, 1–21. <https://doi.org/10.1080/00207543.2025.2502106>
32. Zhang, J., Guo, B., Ding, X., Hu, D., Tang, J., Du, K., ... Jiang, Y. (2024). An adaptive multi-objective multi-task scheduling method by hierarchical deep reinforcement learning. *Applied Soft Computing*, 154, 111342.
33. Khan, A. R. Dynamic load balancing in cloud computing: optimized RL-based clustering with multi-objective optimized task scheduling. *Processes* **12** (3), 519 (2024).
34. Chen, Z., Zhang, L., Wang, X. & Wang, K. Cloud–edge collaboration task scheduling in cloud manufacturing: an attention-based deep reinforcement learning approach. *Comput. Ind. Eng.* **177**, 109053 (2023).
35. Jayanetti, A., Halgamuge, S. & Buyya, R. Reinforcement learning based workflow scheduling in cloud and edge computing environments: A taxonomy, review and future directions. *arXiv preprint arXiv:2408.02938*. (2024).
36. Sun, G., Zhu, G., Liao, D., Yu, H., Du, X., ... Guizani, M. (2019). Cost-Efficient Service Function Chain Orchestration for Low-Latency Applications in NFV Networks. *IEEE Systems Journal*, 13(4), 3877–3888. doi: 10.1109/JSYST.2018.2879883.
37. Sun, G., Liao, D., Zhao, D., Xu, Z. & Yu, H. Live Migration for Multiple Correlated Virtual Machines in Cloud-Based Data Centers. *IEEE Trans. Serv. Comput.*, **11**(2), 279–291. doi: <https://doi.org/10.1109/TSC.2015.2477825> (2018).
38. Song, L., Sun, G., Yu, H. & Niyato, D. ESPD-LP: edge service Pre-Deployment based on location prediction in MEC. *IEEE Trans. Mob. Comput.* **24** (6), 5551–5568. <https://doi.org/10.1109/TMC.2025.3533005> (2025).
39. Du, K., Yang, S., Xu, W., Zheng, F. & Duan, H. A novel optimization framework for efficiently identifying high-quality Pareto-optimal solutions: maximizing resilience of water distribution systems under cost constraints. *Reliab. Eng. Syst. Saf.* **261**, 111136. <https://doi.org/10.1016/j.res.2025.111136> (2025).
40. Li, Z., Gu, W., Shang, H., Zhang, G. & Zhou, G. Research on dynamic job shop scheduling problem with AGV based on DQN. *Cluster Comput.* **28** (4), 236. <https://doi.org/10.1007/s10586-024-04970-x> (2025).
41. Gu, Y., Liu, Z., Dai, S., Liu, C., Wang, Y., Wang, S., ... Cheng, L. (2025). Deep Reinforcement Learning for Job Scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review. *arXiv preprint arXiv:2501.01007*.
42. Zhang, L., Peng, J., Zheng, J. & Xiao, M. Intelligent cloud-edge collaborations assisted energy-efficient power control in heterogeneous networks. *IEEE Trans. Wireless Commun.* **22** (11), 7743–7755 (2023).
43. Liu, S., Xu, N., Li, L., Alharbi, K. H. & Zhao, X. Zero-sum games-based optimal fault tolerant control for control-constrained multiplayer systems with external disturbances via adaptive dynamic programming. *Commun. Nonlinear Sci. Numer. Simul.* <https://doi.org/10.1016/j.cnsns.2025.108804> (2025).
44. Gu, Z., Yan, S., Ahn, C. K., Yue, D. & Xie, X. Event-triggered dissipative tracking control of networked control systems with distributed communication delay. *IEEE Syst. J.* **16**(2), 3320–3330. <https://doi.org/10.1109/JSYST.2021.3079460> (2022).

Author contributions

The research idea and its planning were proposed by all authors, including Wenfan Zhang and Haijiao Ou. The initial draft, ideation, supervision, and coordination of the project were carried out by Haijiao Ou.

Funding

The authors did not receive any financial support for this study.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to H.O.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025